

## تكاليف المقرر

# مترجم باللغة العربية

بناء المترجمات وتصميم لغات البرمجة (Compilers Construction  
& (Programming languages Design)) — CS & IT4

## إشراف:

د. خالد الكحسة

## أسماء الطلاب:

عمران غالب محمد ناصر

محمد احمد حميد اليفرسي

أسامة علي محمد قاسم النشمي

## الفصل الدراسي: الأول

العام الدراسي: 2026 – 2027

# المحتويات

|    |                                                  |
|----|--------------------------------------------------|
| 6  | الفصل الأول — معمارية نظام ACSYS360              |
| 6  | 1.1 — السياق العام والأهداف الأكاديمية           |
| 6  | (C4 Level 1 — System Context)                    |
| 6  | 1.1.1 المشكلة والدافع                            |
| 6  | 1.1.2 التقنيات والأدوات                          |
| 6  | 1.1.3 الأهداف الأكاديمية                         |
| 7  | 1.1.4 مخطط السياق (C4 Level 1)                   |
| 7  | 1.1.5 الجهات المتفاعلة                           |
| 7  | 1.1.6 نطاق المنتج وحدوده                         |
| 8  | 1.1.7 فصل المسؤوليات                             |
| 8  | 2.1 — الحاويات والطبقات المعمارية                |
| 8  | (C4 Level 2 — Containers)                        |
| 8  | 2.1.1 القرار المعماري العام                      |
| 9  | 2.1.2 مخطط الحاويات (C4 Level 2)                 |
| 9  | 2.1.3 طبقات تطبيق Flutter                        |
| 10 | 2.1.4 حاوية Controllerg Presentation             |
| 10 | 2.1.5 حاوية Datag Domain                         |
| 10 | 2.1.6 حاوية compiler_contracts                   |
| 10 | 2.1.7 حاوية arabicc المترجم                      |
| 11 | 3.1 — المكونات التفصيلية لكل حاوية               |
| 11 | (C4 Level 3 — Components per Container)          |
| 11 | 3.1 حاوية Flutter App — المكونات الداخلية        |
| 13 | 3.2 حاوية compiler_contracts — المكونات الداخلية |
| 14 | 3.3 حاوية arabicc — المكونات الداخلية            |
| 18 | 3.4 تحقق البيانات الكامل بين المكونات            |
| 18 | 4.1 — بروتوكول JSON وعقد التكامل                 |
| 18 | (C4 Level 4 — Protocol Contract Deep Dive)       |
| 19 | 4.1.1 الإصدار والفلسفة                           |
| 19 | 4.1.2 دورة حياة الطلب الكاملة                    |
| 19 | 4.1.3 بنية طلب الترجمة — كل حقل بالتفصيل         |
| 20 | 4.1.4 بنية استجابة الترجمة — كل حقل بالتفصيل     |
| 20 | 4.1.5 بنية SourceSpang Diagnostic بالتفصيل       |
| 21 | 4.1.6 بنية SymbolRecordg ProtocolToken           |
| 21 | 4.1.7 نموذج JSON كامل (مثال فعلي)                |
| 22 | 4.1.8 وضع Assist                                 |
| 22 | 4.1.9 الاحترازات والسياسات الكاملة               |
| 23 | 4.1.10 تشغيل المترجم في حزم Desktop              |
| 24 | 5.1 — اللغة العربية: التصميم والمواصفة الكاملة   |
| 24 | 5.1.1 فلسفة التصميم                              |
| 24 | 5.1.2 هيكل البرنامج العام                        |
| 24 | 5.1.3 نظام الأنواع الكامل                        |
| 25 | 5.1.4 التعريفات بالتفصيل                         |

|    |                                                            |
|----|------------------------------------------------------------|
| 26 | 5.1.5 التعليمات الكاملة                                    |
| 27 | 5.1.6 التعبيرات وأولوية العوامل (من lexer.yg parser) ..... |
| 28 | 5.1.7 الوصول إلى البنى المركبة .....                       |
| 28 | 5.1.8 قواعد المعجم التفصيلية .....                         |
| 29 | 5.1.9 أمثلة برامج كاملة .....                              |
| 30 | 5.1.10 الحدود الموثقة للغة .....                           |
| 30 | <b>6.1 — الواجهة الأمامية للمترجم</b>                      |
| 30 | 6.1 المحلل المعجمي (Lexer) .....                           |
| 32 | 6.2 المحلل النحوي (Parser) وبناء AST .....                 |
| 34 | 6.3 شجرة AST — البنية والعقد .....                         |
| 35 | 6.4 إدارة الذاكرة والتشخيصات .....                         |
| 36 | 6.5 حدود الواجهة الأمامية .....                            |
| 36 | <b>7.1 — التحليل الدلالي وجدول الرموز</b>                  |
| 37 | 7.1 الفرق بين النحو والدلالة .....                         |
| 37 | 7.2 جدول الرموز (Symbol Table) .....                       |
| 38 | 7.3 الفحوص التي تُجريها semantic.c .....                   |
| 39 | 7.4 استنتاج نوع التعبير .....                              |
| 39 | 7.5 مسار استدعاء semantic في driver .....                  |
| 40 | 7.6 الحدود الموثقة .....                                   |
| 40 | <b>8.1 — التمثيل الوسيط: 3AC و Typed IR</b>                |
| 40 | 8.1.1 فلسفة Three-Address Code .....                       |
| 40 | 8.1.2 أنواع تعليمات 3AC .....                              |
| 41 | 8.1.3 مثال: تحويل تعبير إلى 3AC .....                      |
| 41 | 8.1.4 تمثيل التحكم في 3AC .....                            |
| 42 | 8.1.5 مخطط تحقق التحويل .....                              |
| 42 | 8.1.6 Typed IR (ملخص بروتوكول JSON) .....                  |
| 43 | 8.1.7 ما يثبت الاختبار الذهبي لـ 3AC .....                 |
| 43 | <b>9.1 — التنفيذ والخلفية وبناء الأثر</b>                  |
| 43 | 9.1.1 مسارا التنفيذ .....                                  |
| 43 | 9.1.2 المفسر الداخلي (Interpreter) .....                   |
| 44 | 9.1.3 مولد NASM x86-64 .....                               |
| 44 | 9.1.4 بناء الأثر (artifact_builder) .....                  |
| 45 | 9.1.5 مثال على Assembly الناتجة .....                      |
| 45 | 9.1.6 ملخص الحدود الأساسية .....                           |
| 46 | <b>10.1 — الاختبارات وضمان الجودة والبناء المستمر</b>      |
| 46 | 10.1.1 استراتيجيات الاختبار .....                          |
| 46 | 10.1.2 اختبارات Smoke Golden .....                         |
| 47 | 10.1.3 تطبيقات الاختبار وما تثبته .....                    |
| 47 | 10.1.4 الأمن والحدود .....                                 |
| 47 | 10.1.5 البناء المستمر (CI) .....                           |
| 48 | 10.1.6 خريطة ملفات المصدر الكاملة .....                    |
| 48 | <b>المراجع والمصادر المعتمدة — الفصل الأول</b>             |
| 48 | المراجع الداخلية (المستودع) .....                          |
| 49 | المراجع الأكاديمية .....                                   |
| 50 | <b>الفصل الثاني — البنية الداخلية للمترجم</b>              |
| 50 | هيكل الفصل .....                                           |

|    |                                                 |
|----|-------------------------------------------------|
| 50 | خريطة المحتوى                                   |
| 51 | المنهجية                                        |
| 51 | <b>2.1 — الصورة العامة ومسار التنفيذ</b>        |
| 51 | 2.1.1 تمهيد                                     |
| 52 | 2.1.2 مخطط المسار العام                         |
| 52 | 2.1.3 جدول مراحل المترجم                        |
| 52 | 2.1.4 المسار ليس سلسلة خطية واحدة               |
| 53 | 2.1.5 دورة الطلب والاستجابة                     |
| 53 | <b>2.2 — التحليل المعجمي (Lexical Analysis)</b> |
| 54 | 2.2.1 ما يفعله Lexer بكلمات بسيطة               |
| 54 | 2.2.2 آلية عمل Flex                             |
| 55 | 2.2.3 الكلمات المدجوزة وأصناف الرموز            |
| 56 | 2.2.4 تحويل الرموز الخاصة                       |
| 56 | 2.2.5 العوامل والترقيم الكامل                   |
| 57 | 2.2.6 قواعد المعجم للأنواع المختلفة             |
| 57 | 2.2.7 معالجة النص العربي UTF-8                  |
| 58 | 2.2.8 التشخيصات المعجمية                        |
| 58 | 2.2.9 مخطط انتقال حالات Lexer المبسط            |
| 59 | <b>2.3 — التحليل النحوي وبناء AST</b>           |
| 59 | 2.3.1 دور المحلل النحوي                         |
| 59 | 2.3.2 نقطة البداية الفعلية                      |
| 59 | 2.3.3 القواعد النحوية الكاملة (EBNF منظم)       |
| 61 | 2.3.4 أسبقية العوامل وارتباطها                  |
| 61 | 2.3.5 عقد AST وأنواعها                          |
| 62 | 2.3.6 بناء AST خطوة بخطوة                       |
| 63 | 2.3.7 أمثلة على أشجار AST لتعليمات مختلفة       |
| 65 | 2.3.8 تشخيصات النحو                             |
| 65 | 2.3.9 حدود التنفيذ الحالي                       |
| 65 | <b>2.4 — التحليل الدلالي ونموذج الأنواع</b>     |
| 65 | 2.4.1 الفرق بين النحو والدلالة                  |
| 66 | 2.4.2 جدول الرموز (Symbol Table)                |
| 67 | 2.4.3 الفحوص التي يُجريها semantic.c            |
| 68 | 2.4.4 استنتاج أنواع التعبيرات                   |
| 68 | 2.4.5 تفريد الأنواع المسماة (Type Resolution)   |
| 68 | 2.4.6 حدود التنفيذ الحالي                       |
| 69 | 2.4.7 مخطط تحقق التحليل الدلالي                 |
| 69 | <b>2.5 — التمثيلات الوسيطة</b>                  |
| 69 | 2.5.1 Three-Address Code (3AC)                  |
| 71 | 2.5.2 Typed IR Summary                          |
| 72 | 2.5.3 العلاقة بين TAC و Typed IRg والمسارات     |
| 72 | <b>2.6 — توليد الشيفرة وبناء الأثر</b>          |
| 72 | 2.6.1 مولد NASM x86-64                          |
| 74 | 2.6.2 حد واضح: تعليمة CALL                      |
| 74 | 2.6.3 بناء الأثر (Artifact Builder)             |
| 75 | 2.6.4 سياسة أمان مسار الأثر                     |
| 75 | 2.6.5 ملاحظة: Assembly ليس ملفاً تنفيذياً       |

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>2.7 — التنفيذ الداخلي والبروتوكول</b>                 | <b>76</b> |
| 2.7.1 المفسر (Interpreter) .....                         | 76        |
| 2.7.2 البروتوكول: الرسالة بين arabicc و Flutter .....    | 77        |
| <b>2.8 — اختبارات التحقق ومصفوفة القبول وحدود الدليل</b> | <b>79</b> |
| 2.8.1 الاختبارات الرئيسية في المستودع .....              | 79        |
| 2.8.2 مصفوفة القبول الشاملة .....                        | 80        |
| 2.8.3 جدول الادعاءات والأدلة .....                       | 80        |
| 2.8.4 قواعد وصف الدعم .....                              | 81        |
| 2.8.5 قيد التحقق في هذه الوثيقة .....                    | 81        |
| <b>2.9 — الخلاصة الهندسية والتوصيات</b>                  | <b>81</b> |
| 2.9.1 ما أُنجز فعلاً .....                               | 81        |
| 2.9.2 مقارنة بالمرحلة القياسية .....                     | 82        |
| 2.9.3 التوصيات لرفع نضج المترجم .....                    | 82        |
| 2.9.4 رسم المسار المستقبلي .....                         | 83        |
| 2.9.5 ملخص الفصل .....                                   | 83        |
| <b>مراجع اللغة المنفذة في مترجم C</b>                    | <b>83</b> |
| 1. الرموز المحجوزة الكاملة .....                         | 83        |
| 2. الفواصل والعوامل .....                                | 84        |
| 3. القواعد النحوية الكاملة (EBNF) .....                  | 85        |
| 4. الأسبقية والارتباط .....                              | 86        |
| 5. القواعد المعجمية .....                                | 87        |
| 6. أمثلة توافقية .....                                   | 87        |
| 7. حدود مثبتة من المصدر .....                            | 87        |
| مصادر التحقق .....                                       | 88        |
| <b>المراجع — الفصل الثاني</b>                            | <b>88</b> |
| المراجع الأكاديمية .....                                 | 88        |
| توثيق الأدوات المستخدمة .....                            | 88        |
| مصادر التحقق من المستودع .....                           | 89        |
| <b>التنفيذ والصور</b>                                    | <b>90</b> |
| واجهه المحرر .....                                       | 90        |
| فتح ملف .....                                            | 90        |
| نتائج ال token .....                                     | 91        |
| جدول الرموز .....                                        | 92        |
| نتائج التحليل الدلالي لخطأ منطقي .....                   | 92        |
| الجدول الرباعي .....                                     | 94        |
| الشفرة التجميعية .....                                   | 94        |

# الفصل الأول — معمارية نظام ACSYS360

## 1.1 — السياق العام والأهداف الأكاديمية

(C4 Level 1 — System Context)

### 1.1.1 المشكلة والدافع

تتبع فكرة المشروع من حاجة تدريس بناء المترجمات إلى بيئة تُظهر أثر كل مرحلة، لا النتيجة النهائية وحدها. لذلك يجمع acsys360 بين محرر لسطح المكتب ومترجم مستقل للغة العربية، بما يتيح للطلاب متابعة:

- Tokens — ناتج التحليل المعجمي - AST — شجرة التحليل النحوي - جدول الرموز — نتيجة التحليل الدلالي - 3AC — التمثيل الوسيط ثلاثي العناوين - Assembly — الشيفرة الآلية النصية - مخرجات التنفيذ — نتائج التشغيل الداخلي

كل هذا ضمن دورة عمل واحدة متكاملة.

### 1.1.2 التقنيات والأدوات

| الأداة/المكتبة | الإصدار        | الغرض                           | القيود                             |
|----------------|----------------|---------------------------------|------------------------------------|
| Flutter        | 3.44.5         | واجهة Desktop اختبارات          | لا ينفذ semantic g grammar         |
| Dart           | مع Flutter     | لغة التطبيق وعقود JSON          | لا يستبدل arabicc                  |
| file_picker    | pubspec.lock   | اختيار ملف/مجلد                 | لا يدبر الحالة                     |
| Flex           | نظام           | توليد scanner من <b>lexer.l</b> | Windows: وضع --wincompat           |
| Bison          | نظام           | توليد parser من <b>parser.y</b> | تحذيرات conflicts ≠ اكتمال grammar |
| CMake          | بناء           | توليد ملفات البناء لكل منصة     | لا يحدد قواعد اللغة                |
| C17            | MSVC/GCC/Clang | بناء arabicc                    | Flutter مستقل عن backend           |
| Cairo font     | assets         | عرض واجهة عربية                 | لا يؤثر على compiler               |

### 1.1.3 الأهداف الأكاديمية

| الهدف                                 | التحقق منه                                |
|---------------------------------------|-------------------------------------------|
| بناء لغة برمجة عربية ذات قواعد محددة  | parser.yg packages/compiler_c/src/lexer.l |
| تنفيذ مراحل الترجمة الكاملة أكاديميًا | مسار compiler_driver_run                  |
| ربط المفاهيم النظرية بالتنفيذ الفعلي  | اختبارات smokeg golden                    |
| محرر Desktop مستقل يعرض نتائج المراحل | تطبيق Flutter في lib/                     |
| عزل الاختبار بين كل مرحلة             | طبقات منفصلة: syntax/semantic/backend     |

المبدأ الجوهرى: كل نتيجة قابلة للتتبع إلى ملفها ومرحلتها. لا تُعلن ميزة بدون تنفيذ واختبار واضحين.

### 1.1.4 مخطط السياق (C4 Level 1)



شكل توضيحي — overview-01

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية. ملاحظة: toolchain (NASM/GCC) يقع خارج حدود التطبيق العادي، ولا يُستدعى إلا عند طلب بناء artifact صريح.

### 1.1.5 الجهات المتفاعلة

| الدور في النظام                       | الجهة/الأداة     |
|---------------------------------------|------------------|
| كتابة وتحليل وتشغيل البرنامج العربي   | المستخدم         |
| واجهة المستخدم وإدارة الحالة والتنسيق | Flutter/Dart     |
| تنفيذ جميع مراحل الترجمة              | arabicc (C)      |
| بناء المترجم وتوليد scanner/parser    | CMake/Flex/Bison |
| بناء artifact عند الحاجة فقط          | NASM/toolchain   |

### 1.1.6 نطاق المنتج وحدوده

ما يشمل النطاق الحالي

- إدارة Workspace وملفات .arb - تحرير النص بالاتجاه RTL وتلوينه معجمياً - التحليل المعجمي والنحوي والدلالي -  
 الإكمال التلقائي والمساعدة عبر --assist - توليد 3AC ونص Assembly - التنفيذ الداخلي عبر interpreter - بناء artifact  
 عند توفر toolchain وتحقق الشروط  
 ما لا يشمل ولا يدّعيه

- Assembly ليست ملفاً تنفيذياً — تحتاج assembler حقيقي - dart-native قيمة في البروتوكول لا مترجم Dart - لا  
 يعني وجود مرحلة احتمال كل تراكيب اللغة فيها - ليس بديلاً عاماً لـ VS Code أو Language Server كامل

## 1.1.7 فصل المسؤوليات

من منظور مقرر بناء المترجمات، تكمن أهمية القرار في فصل concerns:  
 - المكوّن لا يكرر عمل مكوّن آخر - نجاح مرحلة ليس دليلاً تلقائياً على نجاح المراحل الأخرى - يُميّز بين: صحة الصياغة، صحة  
 المعنى، إنتاج التمثيل الوسيط، التنفيذ الداخلي، وبناء الملف الأصلي - الحدود الحالية مُسجّلة صراحةً حتى تبقى النتائج  
 الأكاديمية دقيقة وقابلة للاختبار

## 2.1 — الحاويات والطبقات المعمارية

(C4 Level 2 — Containers)

### 2.1.1 القرار المعماري العام

يعتمد التصميم على عمليتين منفصلتين تماماً:

| الدور                | التقنية        | العملية       |
|----------------------|----------------|---------------|
| الواجهة والتنسيق     | Dart Flutter   | تطبيق Flutter |
| مراحل الترجمة كاملاً | C17 Flex Bison | arabicc       |

لا يوجد FFI بينهما — التطبيق يشغّل المترجم كعملية نظام ويتبادل معه JSON عبر stdin/stdout.  
 هذا الفصل يتيح: - بناء arabicc واختباره مستقلاً عن Flutter - إبقاء تفاصيل Flex/Bison خارج طبقة العرض - إعادة  
 استخدام الملف التنفيذي في حزم Desktop مختلفة (Linux, Windows, macOS)

## 2.1.2 مخطط الحاويات (C4 Level 2)



شكل توضيحي — containers-02

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 2.1.3 طبقات تطبيق Flutter

| الطبقة       | الموقع                           | المسؤولية                                                                                   | الحدود                                            |
|--------------|----------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------|
| Presentation | lib/features/editor/presentation | Widgets, شاشة المحرر, التبويبات, المستكشف, Minimap, لوحات النتائج                           | لا تضع قواعد اللغة ولا تنشئ عملية compiler مباشرة |
| Controller   | .../presentation/controllers     | EditorController — إدارة الحالة المركزية, تنسيق أوامر Workspace, التحرير, التحليل, البناء   | لا تحتوي تفاصيل Flex/Bison                        |
| Domain       | lib/features/editor/domain       | كيانات Workspaceg Document, CompilationResultg, repositories, use cases (بحث, تنسيق, تعليق) | مستقل عن Widgets Flutter UIg                      |
| Data         | lib/features/editor/data         | LocalWorkspaceRepository, ProcessCompilerRepository, workspace, تشغيل عملية arabicc         | لا يقرر قواعد grammar                             |
| Core/Shared  | lib/sharedg lib/core             | ثوابت, أخطاء, خدمات عامة, ثيمات, مكونات مشتركة                                              | لا يربط domain بتفاصيل منصة واحدة                 |

## 2.1.4 حاوية Contollerg Presentation

تعرض طبقة Presentation: - المحرر RTL مع تلوين معجمي - التبويبات (tabs) ووثائق متعددة - مستكشف الملفات (workspace explorer) - لوحات نتائج المراحل Minimap - (Tokens, AST, 3AC, Assembly...) وشريط الحالة يتولى EditorController إدارة: - - compile / buildNative / runNative - changeRoot / refreshFiles / open / save - syncDocument / \_scheduleAnalysis\_ (داخلية) Widgets تستقبل callbacks وتعرض الحالة — لا تنشئ مستودعات أو تتعامل مباشرة مع filesystem.

## 2.1.5 Datag Domain

Domain تضم: - Document — النص الحالي، النسخة المحفوظة، حالة dirty, سجل Workspace - undo/redo — المسار الجذري، الوثائق المفتوحة، التبويب النشط - CompilationResult — نتائج جميع مراحل الترجمة - FileNode — عقدة في شجرة ملفات Workspace Data تنفذ: - قراءة/كتابة الملفات من filesystem - اختيار المجلدات عبر file\_picker - تشغيل عملية arabicc وجمع stdout/stderr - تحويل استجابة JSON إلى نماذج عبر compiler\_contracts

## 2.1.6 compiler\_contracts

حزمة Dart مستقلة في packages/compiler\_contracts/: - تعريف نماذج CompilationRequest CompilationResponse - التحقق من صحة الحقول وإصدار البروتوكول - اختبارات العقد في test/compilation\_protocol\_test.dart - لا تحتوي منطق compiler أو Flutter

## 2.1.7 حاوية المترجم arabicc

الملف التنفيذي arabicc في packages/compiler\_c/:

```
packages/compiler_c/
include/          # C headers
src/
CLI flags — # نقطة الدخول — main.c
  protocol/      # parse/serialize JSON
  driver/        # compiler_driver_run
  lexer.l        # Flex scanner
  parser.y       # Bison parser
# بناء AST وتحريره ast.c
# التحليل الدلالي semantic.c
  ir/            # tac.c typed_ir.c
  runtime/       # interpreter.c
  backend/       # x86_64 artifact toolchain
tests/          # C tests Dart integration
# البناء والتوليد CMakeLists.txt
```

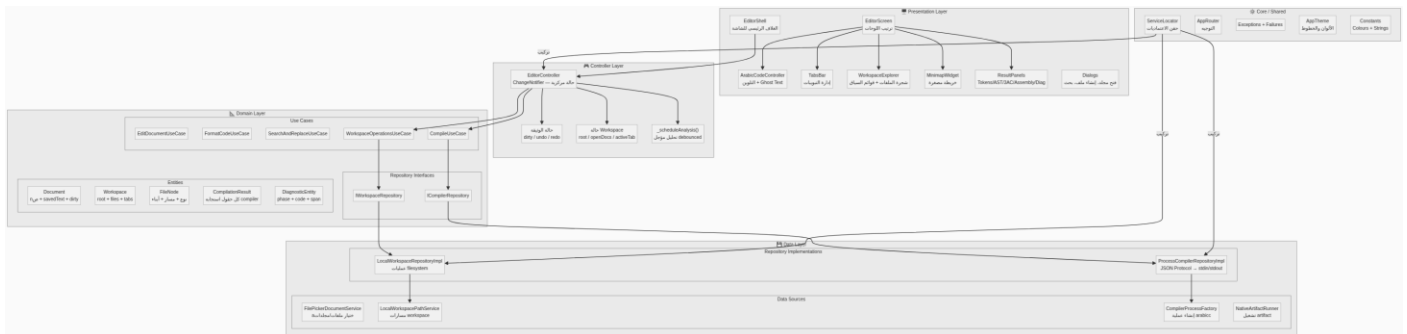
مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 3.1 — المكونات التفصيلية لكل حاوية

(C4 Level 3 — Components per Container)

في نموذج C4 يعني Component مكوناً داخلياً حقيقياً في الحاوية: ملف أو مجموعة ملفات بمسؤولية واضحة ومحددة. هذا القسم يفكك كل حاوية إلى مكوناتها الفعلية مع تدفق البيانات بينها.

### 3.1 Flutter App — المكونات الداخلية



شكل توضيحي — components-03

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

#### 3.1.1 مكونات Presentation

| المكون               | الملف                                      | المسؤولية                                                     |
|----------------------|--------------------------------------------|---------------------------------------------------------------|
| EditorShell          | presentation/ui/screens/editor_screen.dart | الغلاف الرئيسي — يقسم الشاشة إلى مناطق                        |
| ArabicCodeController | presentation/ui/widgets/editor/            | تلوين معجمي ghost text diagnostics underlines                 |
| WorkspaceExplorer    | presentation/ui/widgets/explorer/          | شجرة ملفات RTL, قوائم سياق (إنشاء/فتح/قص/نسخ/حذف/إعادة تسمية) |
| TabsBar              | presentation/ui/widgets/tabs/              | إدارة التبويبات المفتوحة مع علامة dirty                       |
| ResultPanels         | presentation/ui/widgets/panels/            | لوحات Tokens, AST, 3AC, Assembly, Diagnostics                 |
| MinimapWidget        | presentation/ui/widgets/minimap/           | خريطة مصغرة للكود                                             |
| CollapsiblePanel     | shared/widgets/collapsible_panel.dart      | لوحة قابلة للطي — مشتركة                                      |
| Dialogs              | presentation/ui/widgets/dialogs/           | حوارات البحث وإنشاء الملف                                     |

## 3.1.2 مكونات Controller

EditorController

WorkspaceUseCase.open(path) يطلب changeRoot(path)

()WorkspaceUseCase.list يطلب ()refreshFiles

يفتح Document ويضيف تبويماً open(FileNode)

يحفظ Document النشط ()save

Panels ويحدث CompileUseCase يطلب ()compile

buildNative() compile target=dart-native

runNative() compile execute=true

keystroke كل Document تحديث [داخلي] syncDocument(text)\_

debounce [داخلي] ()scheduleAnalysis\_ للتلقائي

[داخلي] تجميع الطلب وإرساله ()compileActive\_

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 3.1.3 مكونات Domain

Domain Layer

Entities/

Document { text, savedText, isDirty, undoStack, redoStack }

Workspace { rootPath, openDocuments[], activeTabIndex }

FileNode { name, path, isDirectory, children[] }

CompilationResult { tokens, syntaxTree, symbolTable, tac,  
assembly, executionOutput, artifacts, diagnostics }

DiagnosticEntity { severity, phase, code, message, span }

Use Cases/

CompileUseCase ICompilerRepository.compile()

WorkspaceOperationsUseCase IWorkspaceRepository.\*

EditDocumentUseCase Document.applyEdit()

تنسيق النص FormatCodeUseCase

بحث واستبدال SearchAndReplaceUseCase

Repository Interfaces/

IWorkspaceRepository { listFiles, createFile, deleteFile, rename, ... }

ICompilerRepository { compile, assist }

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 3.1.4 مكونات Data

Data Layer

Data Sources/

FilePickerDocumentService flutter: file\_picker Dialog اختيار مجلد

Workspace LocalWorkspacePathService يحسب/يتحقق من مسارات

arabicc ل Process ينشئ CompilerProcessFactory  
يشغل الملف التنفيذي الناتج NativeArtifactRunner

Repository Implementations/

LocalWorkspaceRepositoryImpl

```
listFiles(root) File.list() FileNode[]
createFile(path) File.create()
deleteFile(path) File.delete()
rename(old, new) File.rename()
```

ProcessCompilerRepositoryImpl

compile(request)

بناء JSON من CompilationRequest

تشغيل --protocol arabicc

كتابة stdin JSON

قراءة JSON stdout

```
compiler_contracts.parse() CompilationResult
assist(request)
```

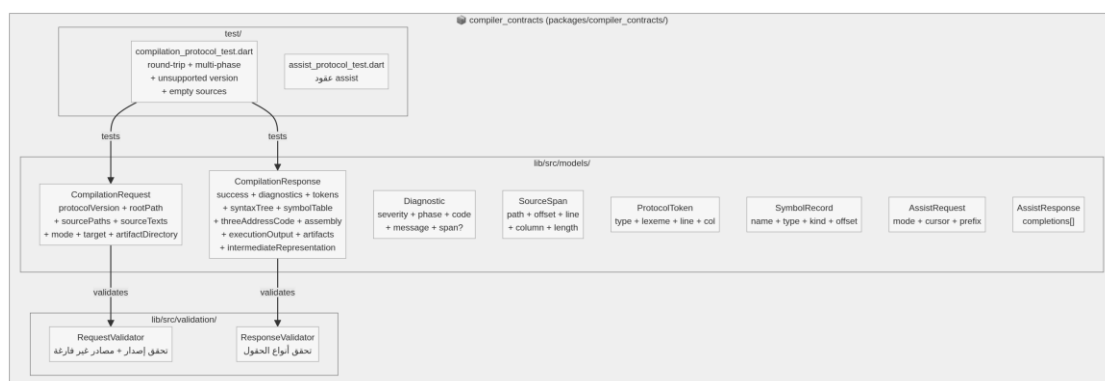
بناء JSON من AssistRequest

تشغيل --assist arabicc

قراءة AssistResponse

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 3.2 حاوية compiler\_contracts — المكونات الداخلية



شكل توضيحي — components-03

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

مسؤولية كل نموذج

| النموذج | الحقول الرئيسية | القيد |
|---------|-----------------|-------|
|---------|-----------------|-------|



`c_serialize_response()` يكتب سطر JSON إلى `stdout`

```
protocol.c
c_run_protocol()
```

يستقبل طلب من `stdin`

يستدعي `compiler_driver_run()`

يرسل استجابة إلى `stdout`

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 3.3.2 مكونات `driver`

```
src/driver/compiler_driver.c
compiler_driver_run(payload, response)
```

1. استخراج مصدر البرنامج من `payload`

2. تهيئة مواضع المصدر (`SourceLocation`)

3. استدعاء `g_root_ast` Flex/Bison

إذا فشل `parser`: إضافة تشخيص `return`

4. تحويل `AST` JSON (`syntax tree`)

5. `c_analyze_semantics()` `CSemanticResult`
6. `c_generate_tac()` `CTacResult`
7. `c_generate_typed_ir()` `intermediateRepresentation`
8. `c_generate_nasm_x86_64()` `assembly text`

إذا `target=dart-native: artifact_build_native()`

(إذا طلب) `executionOutput` ( `9. runtime_execute_ast()`

( تحرير الذاكرة `10. c_ast_free()`

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 3.3.3 مكونات الواجهة الأمامية

```
src/lexer.l (Flex)
```

تعريف `ARABIC_CHAR` نطاقات Unicode للحروف العربية

تعريف `[0-9]` `DIGIT`

قواعد الكلمات المحجوزة (برنامج، ثابت، متغير...)

قاعدة `IDENT` عربي ASCII بعد أول حرف

قواعد الأعداد صحيح وحقيقي

قواعد السلاسل مع `escape sequences`

قواعد المحارف ASCII طباعية عربية

قاعدة الفواصل `','` `':'` `','`

قاعدة التعليق حتى نهاية السطر

تحديث `columnng yylineno` عند كل رمز

```
src/parser.y (Bison)
```

```

... ,token declarations    TOK_PROGRAM, TOK_IF%
تصريحات الأسبقية        left %right%
productions
نقطة البدء الفعلية      full_program
    full_declarations
    full_block
    full_statements
بدائل if/else متعددة   full_if
    full_while
كرر                      full_repeat
full_repeat_until اعد...حتى
full_expression مع أسبقية كاملة
full_access وصول للقوائم والسجلات
()yyerror يسجل التشخيص ويستمر

```

```

src/ast.c include/ast.h
c_ast_node_create(type) تخصيص عقدة
c_ast_node_list_append() إضافة لقائمة
c_ast_node_to_json() تحويل AST إلى JSON
c_ast_free() تحرير الشجرة كاملاً
CTypeSpec primitive | array | record

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 3.3.4 مكونات التحليل الدلالي

```

src/semantic.c
c_analyze_semantics(program_node, result)
declarations تصفح
    constant: أضيف للجدول (kind=CONST)
    variable: أضيف للجدول (kind=VAR)
    type: حلّ النوع المسمى
    procedure: أضيف للجدول (kind=PROC)
statements تصفح
    assignment: تحقق من lvalue تطابق الأنواع
    read: تحقق من تعريف المتغير
    print: اشتق نوع التعبير
    call: ابحث عن الاسم تحقق من التعبيرات
    if/while: فحص تعبير الشرط
    access: حلّ نوع عنصر قائمة أو حقل سجل

infer_type(expression_node, symbols)
    BINARY: حدد النوع من الطرفين والعامل
    UNARY: انقل نوع المعامل
    LITERAL: من قيمة المحرف (INT/REAL/STRING/BOOL/CHAR)
    REFERENCE: ابحث في جدول الرموز

c_semantic_result_free(result)

```

```
include/semantic.h
CSemanticResult { symbols[], diagnostics[] }
SymbolRecord { name, type_kind, symbol_kind, offset, span }
CTypeKind enum { TYPE_INT, TYPE_REAL, TYPE_BOOL, ... }
```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 3.3.5 مكونات التمثيل الوسيط

```
src/ir/tac.c
c_generate_tac(root, result)
gen_expression(node) CTacInstruction* (BINARY/UNARY/ASSIGN)
gen_statement(node)
    ASSIGN    ASSIGN instruction
    IF        BRANCH LABEL JUMP instructions
    WHILE     LABEL BRANCH JUMP
    REPEAT    LABEL counter BRANCH
    REPEAT_UNTIL LABEL BRANCH (
    PRINT     PRINT instruction
    READ      READ instruction
    new_temp() "$t0", "$t1", ...

CTacInstruction { opcode, result, arg1, arg2, op, type, span }
c_tac_result_free()
```

```
src/ir/typed_ir.c
c_generate_typed_ir(ast, semantic, result)
يُنتج: JSON
{ unit, name, target,
  types: [...],
  symbols: [{name, type, offset}...],
  blocks: [{name:"main", statementCount}] }
الإزاحة) offset = i * 8 حساب تسلسلي)
```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 3.3.6 مكونات الخلفية

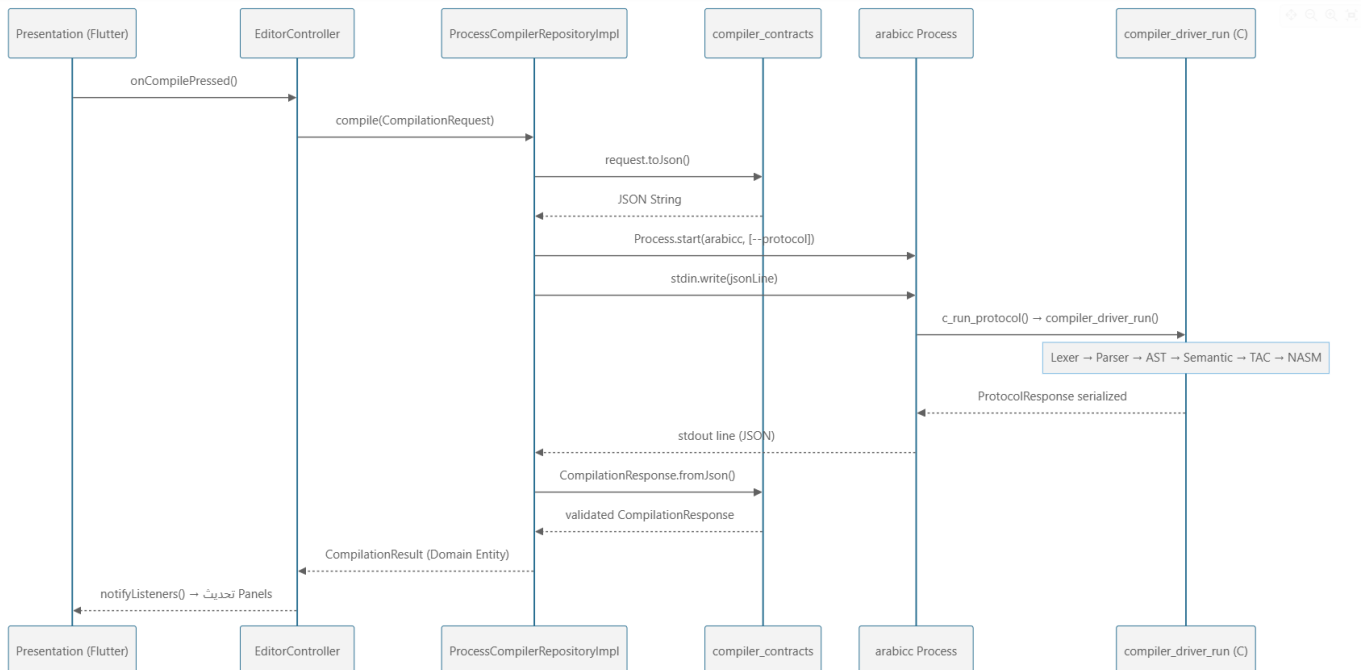
```
src/backend/x86_64/asm_x86_64.c
c_generate_nasm_x86_64(tac, semantic, result)
emit_header()    section .data .text main:
emit_instruction(instr)
    ALLOC    reserve stack space
    ASSIGN   mov instructions
    BINARY   add/sub/imul/ldiv/fadd/...
    UNARY    neg/not
    BRANCH   cmp jcc
    JUMP     jmp
    LABEL    label:
مكتبة C) PRINT    call printf (
مكتبة C) READ     call scanf (
تشفير "لا ABI في" NASM backend CALL ✕
emit_footer()    ret epilogue
```

```
src/backend/artifact_builder.c
artifact_build_native(asm_path, out_path, result)
toolchain_detect() يستدعي
NASM .o object يستدعي
GCC executable يستدعي
out_path < rootPath يتحقق أن
```

```
src/backend/toolchain.c
PATH في GCC و NASM يحدد toolchain_detect()
toolchain_get_nasm_format()
Linux "elf64"
Windows "win64"
toolchain_link(obj, exe) GCC linking
```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 3.4 تدفق البيانات الكامل بين المكونات



شكل توضيحي — components-03

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 4.1 — بروتوكول JSON وعقد التكامل

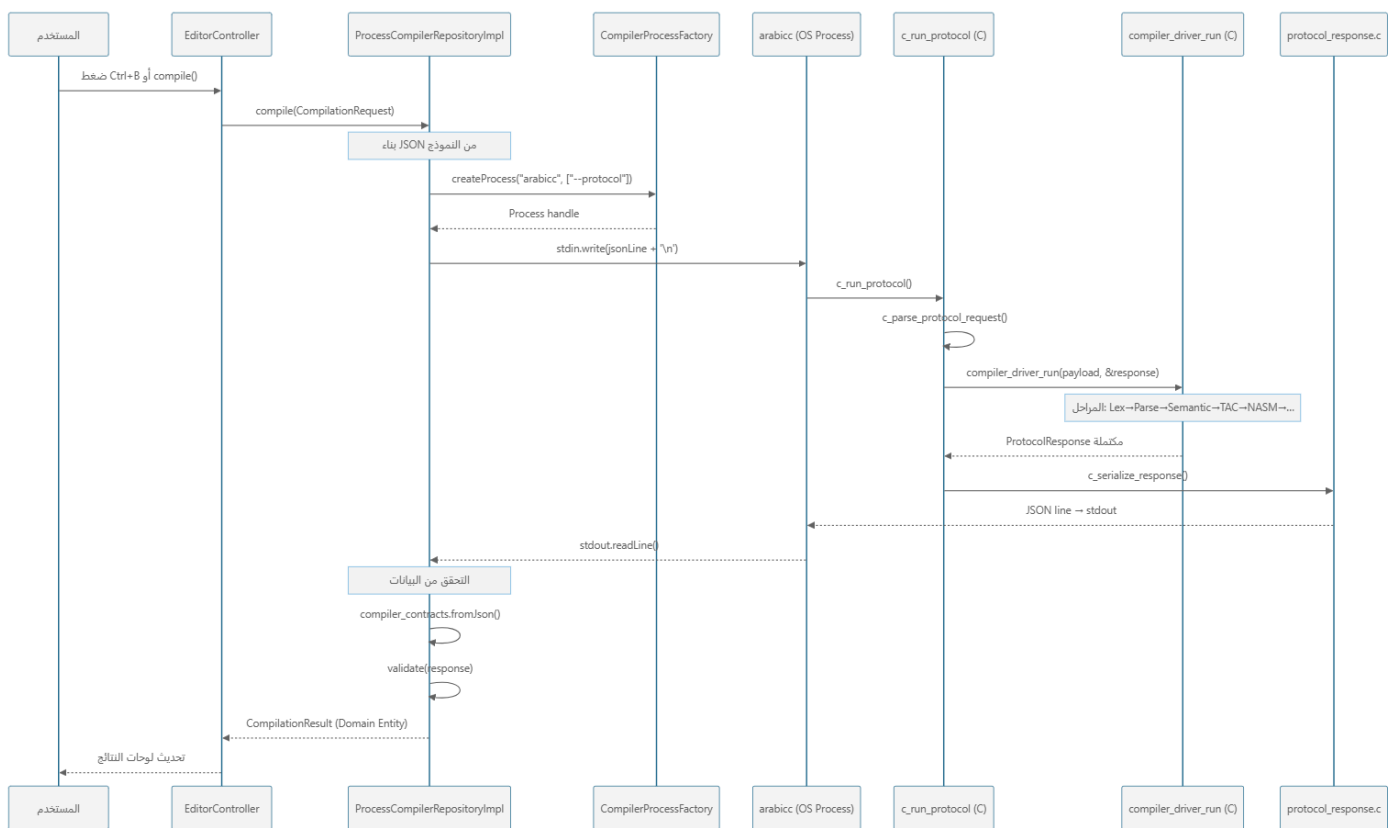
(C4 Level 4 — Protocol Contract Deep Dive)

## 4.1.1 الإصدار والفلسفة

| الخاصية        | القيمة                                         |
|----------------|------------------------------------------------|
| الإصدار الحالي | 0.5.0                                          |
| وسيط الاتصال   | JSON — stdin/stdout سطر واحد                   |
| مصدر النماذج   | packages/compiler_contracts/                   |
| قاعدة الرفض    | إصدار غير مدعوم رسالة خطأ واضحة, لا تفسير صامت |

المحرر والمترجم عمليتان منفصلتان تماماً — لا FFI, لا shared memory, لا مكتبة مشتركة.

## 4.1.2 دورة حياة الطلب الكاملة



شكل توضيحي — protocol-04

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 4.1.3 بنية طلب الترجمة — كل حقل بالتفصيل

```

CompilationRequest {
  — protocolVersion: String "0.5.0" إلزامي، أي قيمة أخرى = رفض
  — rootPath: String جذر Workspace إلزامي، مسار مطلق
  — sourcePaths: List<String> مسارات — .arb إلزامي، لا تكون فارغة
  — sourceTexts?: Map<String,String> النصوص (أولوية على القرص)
}
  
```

```

: "project" | "active" | "project" (
  mode?: String
  entryPath?: String
  target?: String "none" | "dart-native"
  artifactDirectory?: String
  artifact: String
  rootPath: String
  execInput?: List<String>
}

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

قواعد التحقق في RequestValidator:

✓ "protocolVersion == "0.5.0" وإلا: خطأ "إصدار غير مدعوم"

✓ rootPath != null && rootPath == "" وإلا: خطأ "مسار جذر مفقود"

✓ sourcePaths.isNotEmpty() وإلا: خطأ "قائمة مصادر فارغة"

✓ كل مسار في sourcePaths يقع تحت rootPath

✓ artifactDirectory (إن وُجد) يقع تحت rootPath

✓ mode ∈ {"active", "project"} أو null

✓ target ∈ {"none", "dart-native"} أو null

#### 4.1.4 بنية استجابة الترجمة — كل حقل بالتفصيل



#### 4.1.5 بنية SourceSpan Diagnostic بالتفصيل

```

Diagnostic {
  severity: "error" | "warning" | "info"
  phase: "lexer" | "parser" | "semantic" | "backend" | "runtime" | "artifact"
  code: String
  message: String
  span?: SourceSpan
}

SourceSpan {
  sourcePath: String
  offset: int
  line: int
  column: int
  length: int
}

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## أمثلة رموز التشخيص:

| الرمز | المرحلة  | المعنى                          |
|-------|----------|---------------------------------|
| L001  | lexer    | رمز غير معروف                   |
| P001  | parser   | خطأ نحوي في التركيب             |
| S001  | semantic | تعريف مكرر                      |
| S002  | semantic | مرجع غير معرف                   |
| S003  | semantic | تعديل ثابت                      |
| S004  | semantic | عدم توافق الأنواع               |
| B001  | backend  | ABI غير منفذ لـ CALL            |
| A001  | artifact | artifactDirectory خارج rootPath |

## 4.1.6 بنية SymbolRecordg ProtocolToken

```
ProtocolToken {
  : "TOK_PROGRAM", "TOK_IDENT", ... اسم الرمز. type: String
  lexeme: String النص الأصلي كما ظهر في المصدر
  line: int رقم السطر(من 1)
  column: int رقم العمود(من 1)
  offset: int إزاحة بالبايتات(من 0)
  length: int طول الرمز بالبايتات
}

SymbolRecord {
  name: String اسم المعرف
  type: String "صحيح"|"حققي"|"منطقي..."
  kind: String "variable"|"constant"|"procedure"|"type"
  offset: int إزاحة في جدول الرموز(تسلسلية 8*8)
  span: SourceSpan موضع التعريف
}
```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 4.1.7 نموذج JSON كامل (مثال فعلي)

## الطلب

```
{
  "protocolVersion": "0.5.0",
  "rootPath": "C:/projects/arabic",
  "sourcePaths": ["C:/projects/arabic/main.arb"],
  "sourceTexts": {
    "C:/projects/arabic/main.arb":
    "\n.\n":)أ(اطبع\n : = 2 3أ\n{\n : صحيح : متغير أ : صحيح :
  },
  "mode": "active",
  "target": "none"
}
```

## الاستجابة

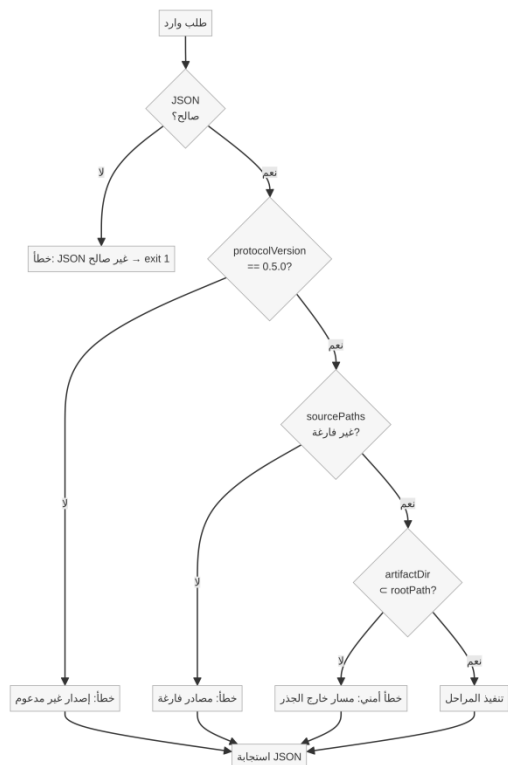
```
{
  "success": true,
```

```

"diagnostics": [],
"tokens": [
  {"line":1,"column":1,"offset":0,"length":7,"type":"TOK_PROGRAM","lexeme":"برنامج",
  {"line":1,"column":9,"offset":8,"length":5,"type":"TOK_IDENT","lexeme":"حساب",
  {"line":1,"column":14,"offset":13,"length":2,"type":"TOK_SEMI","lexeme":";"},
],
"symbolTable": [
  {"kind":"variable","offset":8,"صحيح","type":"i" {"name":"i",
    "span":{"sourcePath":".../main.arb","offset":15,"line":2,"column":8,"length":1}}
],
"threeAddressCode": [
  "ALLOC",
  "$t0 = 2 3",
  "t0$ = I",
  "PRINT",
],
"intermediateRepresentation": {
  "target":"none","حساب":"unit":"main","name":
  "صحيح" "types":[
    {"offset":8}], صحيح, "type":"i" "symbols":[{"name":"i",
    "blocks":[{"name":"main","statementCount":2}]
  },
  "assembly": "section .data\n fmt_int db \"%d\", 10, 0\nsection .text\n global main\n extern printf\nmain:\n push rbp\n mov rbp, rsp\n mov eax, 2\n add eax, 3\n mov [rbp-8], eax\n lea rdi, [fmt_int]\n mov esi, [rbp-8]\n call printf\n pop rbp\n ret\n",
  "executionOutput": ["5"],
  "artifacts": []
}

```

## 4.1.8 Assist وضع



```
arabicc --assist
```

```

AssistRequest {
  protocolVersion: "0.5.0"
  mode: "completion" | "hover" | "signature"
  cursor: { line, column, offset }
  prefix: String
  sourceText: String
}

```

```

AssistResponse {
  completions: List<CompletionItem>
  hover?: HoverInfo
}

```

```

CompletionItem {
  label: String
  kind: String "keyword" | "identifier" | "snippet"
  detail: String
}

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

شكل توضيحي — protocol-04

قائمة الكلمات في protocol\_request.c هي كتالوج للمساعدة فقط، وليست مرجع القواعد — المرجع هو lexer.

## 4.1.9 الاحترازات والسياسات الكاملة

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل

مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

| القاعدة                         | التطبيق                                      |
|---------------------------------|----------------------------------------------|
| إصدار غير مدعوم                 | استجابة JSON بـ success=false                |
| مصادر فارغة                     | تشخيص فوري لا تنفيذ                          |
| artifactDirectory خارج rootPath | artifact_builder.c يرفض ويُضيف diagnostic    |
| استجابة غير صالحة JSON          | لا تُحوّل إلى نجاح صوري — exit code ≠ 0      |
| timeout للعملية                 | ProcessCompilerRepositoryImpl يفرض حداً أقصى |
| إلغاء إدخال تفاعلي              | خطأ runtime صريح — لا قيمة صفر افتراضية      |

## 4.1.10 تشغيل المترجم في حزم Desktop

```

بيئة التطوير
Linux/macOS: packages/compiler_c/build/arabicc
Windows: packages/compiler_c/build/Release/arabicc.exe

حزمة Release
Linux/macOS: <AppDir>/compiler/arabicc
Windows: <AppDir>/compiler/arabicc.exe

البحث بالترتيب:
1. compiler (مضمّن في bundle)
2. مسار التطوير (fallback للتطوير فقط)

Windows Release = Inno Setup installer يحتوي:
المترجم arabicc.exe
التطبيق acsys360.exe
flutter_windows.dll runtime Flutter
أصول Flutter /data
compiler/arabicc.exe المترجم مضمّن
msys2-ucrt64/ GCC NASM لبناء artifacts
  
```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 5.1 — اللغة العربية: التصميم والمواصفة الكاملة

### 5.1.1 فلسفة التصميم

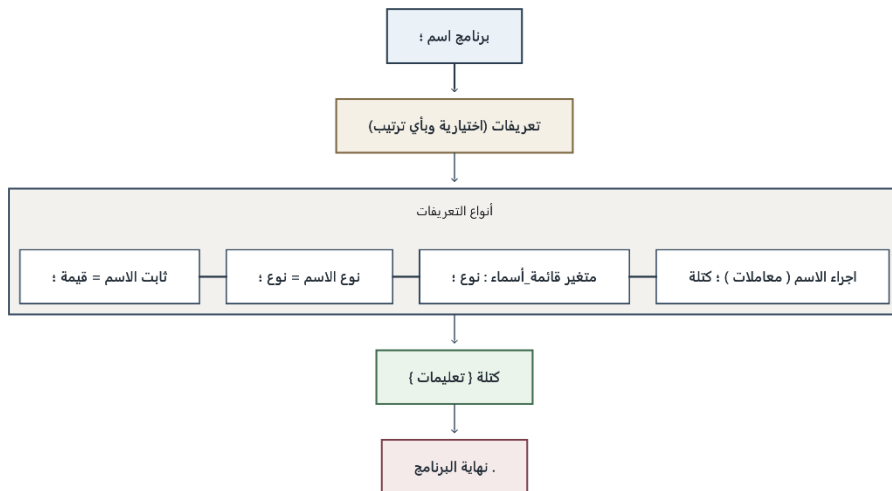
اللغة العربية ليست ترجمة حرفية لـ Pascal أو C  
↓  
كلماتها المحجوزة عربية أصيلة  
↓  
تراكيبها تعكس نمط التعبير العربي  
↓  
Lexer يقبل بعض البدائل الإملائية (همزة / بلا همزة)  
↓  
المعرّفات يمكن أن تكون عربية (أسماء ذات معنى)

القاعدة الذهبية: اللغة تُعرّف بما في `lexer.yg`, لا بالأسماء أو الخطط.

### 5.1.2 هيكل البرنامج العام

```
برنامج اسم_البرنامج ؛
[ تعريفات [ ثوابت، أنواع، متغيرات، إجراءات
{
  تعليمات
}.
```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر



شكل توضيحي — language-05

### 5.1.3 نظام الأنواع الكامل

(CTypeKind الأنواع الممكنة)

الأنواع البدائية

صحيح (32-bit) int في (backend)

حقيقي float/double  
 منطقي bool: صح | خطأ  
 حرفي (UTF-8) char  
 خيط\_رمزي string نص بين ("...")

الأنواع المركبة  
 قائمة [ N ] من نوع مصفوفة بحجم ثابت  
 N: عدد صحيح literal أو معرّف (معرّف length=0 حالياً)  
 سجل { حقول : نوع struct: { ... }

نوع مسقى IDENT يُحلّ لاحقاً بواسطة semantic

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر  
 التوافق بين الأنواع (semantic.c):

مصدر هدف النتيجة  
 صحيح حقيقي ✓ مسموح  
 حقيقي صحيح X مرفوض  
 أي منطقي X صراحة  
 صحيح صحيح ✓  
 نوع مسمى نوع أصلي يُحلّ أولاً

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 5.1.4 التعريفات بالتفصيل

الثابت

ثابت الاسم = تعبير ؛

أمثلة:

ثابت أقصى = 100 ؛

ثابت رسالة = "مرحباً" ؛

ثابت نسبة = 3.14 ؛

المتغير

متغير الاسم : نوع ؛

متغير اسم 1 , اسم 2 , اسم 3 : نوع ؛ أسماء متعددة من نوع واحد

أمثلة:

متغير عدد : صحيح ؛

متغير س , ص , ع : حقيقي ؛

متغير نص : خيط\_رمزي ؛

## مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### نوع القائمة

نوع الاسم = قائمة [ حجم ] من نوع\_العنصر ؛

أمثلة:

نوع مصفوفة = قائمة [ 10 من صحيح ؛

نوع نصوص = قائمة [ 5 من خيط\_رمزي ؛

### نوع السجل

نوع الاسم = سجل {

حقول: 1 نوع ؛

حقول: 2 نوع ؛

حقول: 3 نوع ؛

؛ }

مثال:

نوع نقطة = سجل {

س : حقيقي ؛

ص : حقيقي

؛ }

### الإجراء

إجراء الاسم ( معاملات ) {

تعليمات

}

المعاملات:

بالقيمة اسم : نوع

بالمرجع اسم : نوع

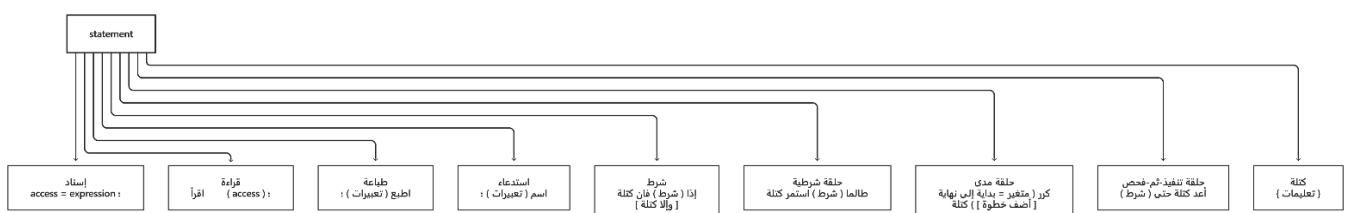
بالقيمة أ , ب : صحيح ؛ بالمرجع نتيجة : صحيح مجموعة متعددة

قيد حالي NASM backend: لا يملك ABI للإجراءات تشخيص B001

قيد حالي interpreter: لا ينفذ CALL حالياً

## مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 5.1.5 التعليمات الكاملة



### شكل توضيحي — language-05

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل

مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

صيغ if المتعددة (parser.y)

صيغة 1: if بدون else

إذا ( شرط ) فان تعليمة ؛

صيغة 2: if مع else

إذا ( شرط ) فان تعليمة ؛ وإلا تعليمة ؛

صيغة 3: if-else if (تسلسل)

إذا ( شرط 1 ) فان تعليمة ؛

وإلا إذا ( شرط 2 ) فان تعليمة ؛

وإلا تعليمة ؛

صيغ while

صيغة أساسية

طالما ( شرط ) استمر تعليمة

مع كتلة

{ طالما ( شرط ) استمر {

تعليمات

}

مع فاصلة قبل الكتلة

{ طالما ( شرط ) ؛ استمر { تعليمات }

صيغ repeat-until

مع كتلة

{ اعد

تعليمات

{ حتى ( شرط ) ؛

تعليمة واحدة

اعد تعليمة حتى ( شرط ) ؛

كتلة فارغة

اعد حتى ( صح ) ؛

## 5.1.6 التعبيرات وأولوية العوامل (من lexer.l إلى parser.y)

أولوية العوامل (من الأدنى إلى الأعلى):

1. || (أو منطقي) يساري
2. && (وأند منطقي) يساري
3. != < > <= >= يساري
4. - (جمع وطرح) يساري
5. % \ (ضرب, قسمة, باقي, صحيحة) يساري
6. ^ (أس) يميني
7. - ! (أحادية) يميني (UMINUS/UPLUS)

الرموز والبدائل المقبولة:

| العامل           | الصيغة   | ملاحظة          |
|------------------|----------|-----------------|
| القسمة الصحيحة   | \ أو \   | كلاهما رمز واحد |
| أقل أو يساوي     | >= أو >  | كلاهما مقبولان  |
| أكبر أو يساوي    | = أو <=  | كلاهما مقبولان  |
| لا يساوي         | != أو != | كلاهما مقبولان  |
| الفاصلة المنقوطة | ! أو ;   | تُحوّل داخلياً  |
| الفاصلة          | , أو ,   | تُحوّل داخلياً  |

### 5.1.7 الوصول إلى البنى المركبة

```
access ::= IDENT { "[" expression "]" | "." IDENT }
```

أمثلة:

أرقام  
[0]عنصر قائمة  
نقطة.س  
حقل سجل  
مصفوفة.[i]الاسم  
قائمة [i][j]مصفوفة ثنائية (إذا دعمها parser)

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 5.1.8 قواعد المعجم التفصيلية

المعرّف:

يبدأ بـ: [عربي (ARABIC\_CHAR)] | [A-Za-z] | \_  
يتبعه: [عربي] | [A-Za-z0-9] | \_

ARABIC\_CHAR (نطاقات UTF-8 في lexer.1):

الحروف العربية الأساسية \xD8\x80–\xD9\xBF  
المناطق العربية الممتدة \xFB\x50–\xFD\xFF

الأعداد:

صحيح: [9-0]

حقيقي: [9-0] ":" [9-0]

(الإشارة – أو عامل أحادي في parser, ليست جزءاً من literal)

النصوص:

سلسلة: "" (أي محرف | \ أي\_محرف) \*

محرف: " محرف واحد "

أو بالاقتراسات الطباعية العربية: '...'

التعليق:

حتى نهاية السطر

يُصدّر ضمن tokens للعرض, لا يُرجع رمزاً نحوياً

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 5.1.9 أمثلة برامج كاملة

مثال 1: حساب بسيط

برنامج حساب ؛

متغير أ , ب , ج : صحيح ؛

{

اقرا ( أ ) ؛

اقرا ( ب ) ؛

ج = أ \* ب \* 2 ؛

اطبع ( ج ) ؛

}.

مثال 2: شرط متداخل

برنامج تصنيف ؛

متغير درجة : صحيح ؛

{

اقرا ( درجة ) ؛

إذا ( درجة <= 90 ) فان

اطبع " ممتاز " ؛

وإلا إذا ( درجة <= 75 ) فان

اطبع " جيد جداً " ؛

وإلا إذا ( درجة <= 60 ) فان

اطبع " جيد " ؛

وإلا

اطبع " راسب " ؛

}.

## مثال 3: حلقة مدى مع قائمة

```

برنامج مجموع ؛
نوع أرقام = قائمة [ ٥ من صحيح ؛
متغير بيانات : أرقام ؛
متغير مج , ع : صحيح ؛
{
    مج = 0 ؛
    كرر ( ع = 0 الى 4 أضف ) 1
    اقرا ( بيانات [ ع ] ) ؛
    مج = مج + بيانات [ ع ] ؛
}
اطبع " ( المجموع = " , مج ) ؛
.

```

## مثال 4: سجل ومعاملات

```

برنامج سجلات ؛
نوع طالب = سجل {
    اسم : خيط_رمزي ؛
    درجة : حقيقي ؛
}
متغير ط : طالب ؛
{
    ط.درجة = 85.5 ؛
    اطبع ( ط.درجة ) ؛
}
.

```

## 5.1.10 الحدود الموثقة للغة

| القيد                      | السبب                                | الأثر                               |
|----------------------------|--------------------------------------|-------------------------------------|
| الإجراءات بدون ABI في NASM | CALL تشخيص B001                      | لا native code للإجراءات            |
| interpreter لا ينفذ CALL   | حالة غير موجودة في execute_statement | التنفيذ الداخلي للإجراءات محدود     |
| طول القائمة بمعرف          | CTypeSpec.length = 0                 | قبول نحوي فقط, لا تقييم وقت الترجمة |
| نطاق مشترك للإجراءات       | جدول رموز خطي واحد                   | لا عزل بين متغيرات الإجراءات        |
| اعد...حتى                  | ليست في execute_statement بالمفسر    | التنفيذ الداخلي لا يدعمها           |

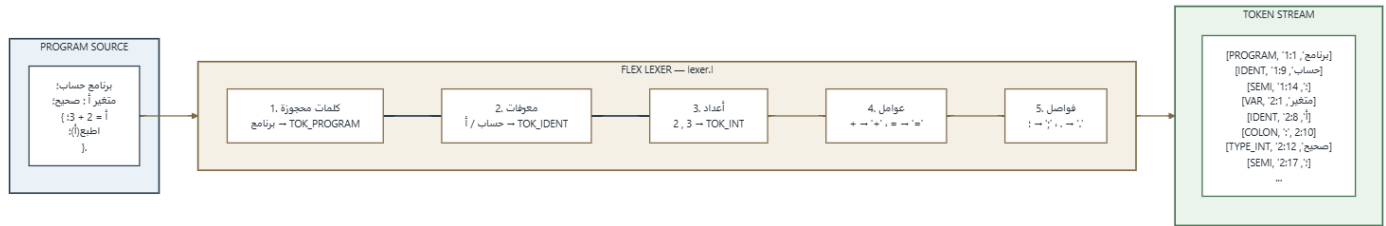
## 6.1 — الواجهة الأمامية للمترجم

الواجهة الأمامية هي الجزء الذي يفهم اللغة. مهمتها تحويل النص العربي الخام إلى بنية بيانات منظمة (AST) يستطيع بقية المترجم التعامل معها. تتكون من ثلاثة مكونات متسلسلة.

## 6.1 المحلل المعجمي (Lexer)

ما يفعله بكلمات بسيطة

تخيّل أنك تقرأ جملة عربية — أول شيء يفعله دماغك أنه يُقسّم النص إلى كلمات ورموز قبل أن يفهم معناها. هذا بالضبط ما يفعله Lexer: يقرأ النص حرفاً حرفاً ويقول "هذه كلمة محبوزة، هذا معرّف، هذا رقم، هذا عامل".



شكل توضيحي — frontend-06

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكوّن ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

كيف يختار Lexer القاعدة الصحيحة

قاعدة أطول تطابق: إذا تطابق النص مع أكثر من قاعدة، يختار Lexer دائماً الأطول. مثلاً كلمة برنامج تتطابق مع قاعدة "كلمة محبوزة" وقاعدة "معرّف"، فيختار "كلمة محبوزة" لأنها أكثر تحديداً وتأتي أولاً في الملف.

lexer.l — ترتيب القواعد مهم:

1. كلمات محبوزة (برنامج، ثابت...) أولاً لمنع مطابقتها كمعرّفات
2. معرفّات ( أي اسم عربي/ASCII)
3. أعداد حقيقية (3.14) قبل الصحيحة (أطول)
4. أعداد صحيحة ( 42)
5. نصوص ( "مرحبا")
6. محارف ( 'أ')
7. عوامل وفواصل ( , , - , : , ...)
8. تعليقات ( ... ) تُتجاهل نحوياً
9. رمز مجهول L001 تشخيص

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

كيف يعالج Lexer النص العربي

المحرف العربي يشغل 2 إلى 4 بايت في UTF-8، لذلك يتتبع Lexer موضعين: - offset — الإزاحة بالبايتات (للبروتوكول) - column — رقم العمود بالمحارف (للعرض للمستخدم)

نص: "برنامج"

( offset: 0 يبدأ من بداية الملف)

14 bytes: (7 محارف × 2 بايت لكل حرف عربي)

line: 1  
col: 1

الرموز الخاصة التي يحوّلها Lexer

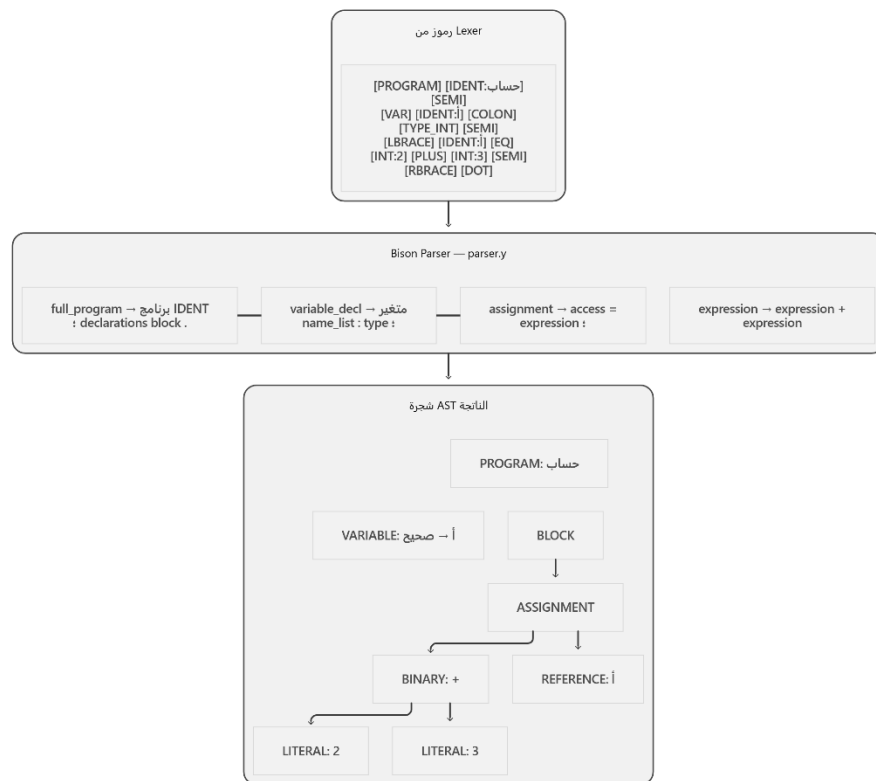
| الرمز المكتوب | الرمز الداخلي | السبب |
|---------------|---------------|-------|
|---------------|---------------|-------|

|          |           |                |
|----------|-----------|----------------|
| ؛ (عربي) | ; (ASCII) | توحيد المعالجة |
| , (عربي) | , (ASCII) | توحيد المعالجة |
| =<       | TOK_LE    | بدل ل =>       |
| =>       | TOK_GE    | بدل ل =<       |
| =!       | TOK_NE    | بدل ل !=       |

## 6.2 المحلل النحوي (Parser) وبناء AST

ما يفعله بكلمات بسيطة

بعد أن ينتهي Lexer من تقطيع النص إلى كلمات، يأتي Parser ليسأل: هل هذه الكلمات مرتبة ترتيباً منطقياً يُشكّل جملاً صحيحة؟ مثلاً متغير أ : صحيح جملة صحيحة، لكن أ متغير : صحيح خاطئة. وعند قبول الجملة، يبني Parser شجرة (AST) تمثل البنية الهرمية للبرنامج.



شكل توضيحي — frontend-06

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكوّن ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

نقطة البداية الفعلية في parser.y

```

program
نقطة البداية الحقيقية ► full_program
" برنامج " IDENT "؛"
{ declaration " : { "صفر أو أكثر"

```

block "."

( القواعد القديمة مثل statement و expr موجودة في الملف لكنها لا تُوصَل من full\_program )

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر  
كيف تُبنى الشجرة خطوة بخطوة

الإدخال:  $2 = 3$  ;

خطوة 1: Parser يرى IDENT:

يتعرف على C\_AST\_REFERENCE access ("أ")

خطوة 2: يرى '='

يبدأ assignment

خطوة 3: يرى INT:

C\_AST\_LITERAL(2)

خطوة 4: يرى ''

ينتظر الطرف الأيمن

خطوة 5: يرى INT:

C\_AST\_LITERAL(3)

خطوة 6: يبني العملية الثنائية

C\_AST\_BINARY\_EXPR(' ', literal(2), literal(3))

خطوة 7: يكمل الإسناد

ref ("أ"), (2,3), (binary(' ',2,3))

C\_AST\_ASSIGNMENT(

أسبقية العوامل (تصريحات Bison)

من الأدنى إلى الأعلى:

( || أو منطقي ) — يساري

&& ( و منطقي ) — يساري

== != < > <= >= ( مقارنة ) — يساري

- ( جمع وطرح ) — يساري

\*/ \% ( ضرب وقسمة ) — يساري

^ ( أس ) — يميني

( - ! أحادية ) — يميني الأعلى

مثال:  $2 * 2$

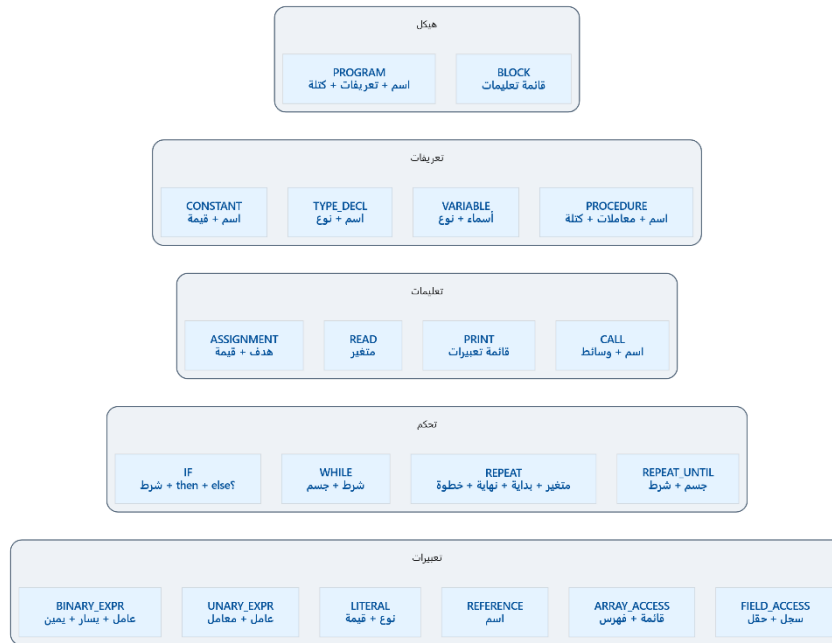
يُحسب ب \* 2 أولاً (أعلى أسبقية)  
ثم أ النتيجة

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 6.3 شجرة AST — البنية والعقد

ما هي AST؟

AST (Abstract Syntax Tree) هي التمثيل الرسمي للبرنامج في الذاكرة. كل عقدة تمثل بنية من اللغة. المراحل التالية (Semantic, TAC, Interpreter) تعمل جميعها على هذه الشجرة.



#### شكل توضيحي — frontend-06

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

بنية عقدة AST في C

كل عقدة في الشجرة من هذا النوع

```
typedef struct CAstNode {
```

```
    نوع العقدة CAstNodeType type;
```

```
    الاسم ( للمتغيرات, الإجراءات,...) char* name;
```

```
    struct CAstNode* left; // (الثنائية للعمليات) الأيسر الفرع
    struct CAstNode* right; // الأيمن الفرع
```

```
    struct CAstNode* condition; // والحلقات للشرط
    struct CAstNode* then_branch; // then فرع
    struct CAstNode* else_branch; // else فرع (اختياري)
    struct CAstNode* body; // الحلقة أو الإجراء جسم
```

children \* CastNodeList; قائمة تعليمات الكتلة

params; \* CastNodeList معاملات الإجراء

type\_spec; \* CTypeSpec النوع (للمتغيرات والثوابت)

للقيم الحرفية

literal; union { int i; double d; char\* s; char c; int b; }

الموضع في النص الأصلي

column; line, offset; int CastNode;

مثال: AST مرئي لبرنامج حقيقي

البرنامج:

: مثال برنامج

: صحيح: ب, أ متغير

{

: 5 = أ

: 2 \* أ = ب

: (ب) اطبع

}.

الشجرة:

"مثال" PROGRAM

[declarations]

VARIABLE

names: ["أ", "ب"]

type: صحيح

[body] BLOCK

ASSIGNMENT

target: REFERENCE "أ"

value: LITERAL 5

ASSIGNMENT

target: REFERENCE "ب"

value: BINARY\_EXPR '\*\*

REFERENCE "أ"

LITERAL 2

PRINT

args: [REFERENCE "ب"]

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 6.4 إدارة الذاكرة والتشخيصات

إدارة الذاكرة

C لا توفر Garbage Collector — كل ما يُخصَّص يجب أن يُحرَّر يدوياً. driver يتولى هذا بالترتيب:

compiler\_driver\_run()

root \* CastNode yyparse() [تخصيص الشجرة]

... تنفيذ المراحل ...

[يملأ الاستجابة] c\_build\_protocol\_response()

[تحرير الشجرة] c\_ast\_free(root)

c\_semantic\_result\_free(...)

c\_tac\_result\_free(...)

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### جدول التشخيصات

| الكود | المرحلة | السبب                | المثال                      |
|-------|---------|----------------------|-----------------------------|
| L001  | Lexer   | رمز غير معروف        | @ أو محرف لا يطابق أي قاعدة |
| L002  | Lexer   | سلسلة نصية غير مغلقة | "مرحباً بلا إغلاق"          |
| P001  | Parser  | خطأ نحوي             | متغير = 5 بدون اسم          |
| P002  | Parser  | رمز غير متوقع        | كلمة محجوزة في مكان خاطئ    |

## 6.5 حدود الواجهة الأمامية

✓ ما تثبته:

النص العربي يُقرأ ويُنتج tokens

البرامج الصحيحة تُبنى كـ AST

الأخطاء المعجمية والنحوية تُشخص بمواضع دقيقة

التحليل يستمر بعد الخطأ (لا يتوقف عند أول مشكلة)

X ما لا تثبته وحدها:

أن الأسماء معرّفة مسؤولية semantic

أن الأنواع متوافقة مسؤولية semantic

أن الإجراءات قابلة للتنفيذ native مسؤولية backend

## 7.1 — التحليل الدلالي وجدول الرموز

التحليل الدلالي هو المرحلة التي تسأل: هل البرنامج منطقي؟ يمكن للجملة أن تكون صحيحة نحوياً تماماً ولكنها بلا معنى — مثلاً تعديل ثابت، أو جمع رقم مع نص. هنا يتدخل semantic.c.

## 7.1 الفرق بين النحو والدلالة



شكل توضيحي — semantic-07

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكوّن ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 7.2 جدول الرموز (Symbol Table)

ما هو؟

جدول الرموز هو سجل لكل اسم ظهر في البرنامج؛ من أين جاء، ما نوعه، هل هو ثابت أم متغير أم إجراء.

البرنامج المصدر:  
ثابت أقصى = 100 ;  
متغير عدد : صحيح ;  
متغير نسبة : حقيقي ;  
إجراء احسب (...):

Symbol Table — CSemanticResult

| name | kind      | type  | offset |
|------|-----------|-------|--------|
| أقصى | constant  | صحيح  | 8      |
| عدد  | variable  | صحيح  | 16     |
| نسبة | variable  | حقيقي | 24     |
| احسب | procedure | —     | 32     |

شكل توضيحي semantic-07

## البنية في C

```
typedef struct {
    char*   name;   اسم الرمز
    CTypeKind type_kind; ... صحيح | حقيقي | منطقي | ...
    CSymKind sym_kind;   variable | constant | procedure | type
    int      offset;     إزاحة = 8 * index
    SourceSpan span;     موضع التعريف في المصدر
} SymbolRecord;

typedef struct {
    SymbolRecord* symbols; مصفوفة الرموز
    int          count;
    Diagnostic*  diagnostics; أخطاء التحليل
    int          diag_count;
```

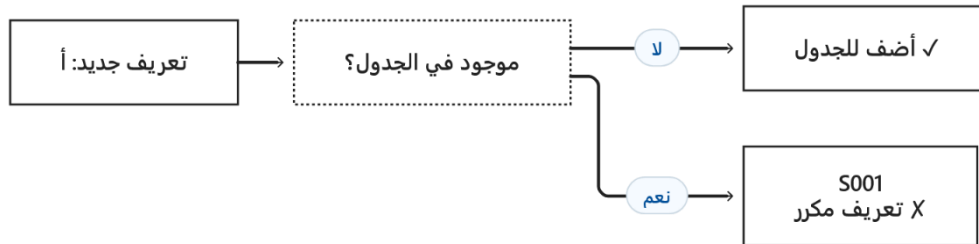
### 7.3 الفحوص التي يُجريها semantic.c

#### الفحص 1: تكرار التعريف

متغير أ: صحيح؛

متغير أ: حقيقي؛ خطأ S001: الاسم "أ" معرّف مسبقاً

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر



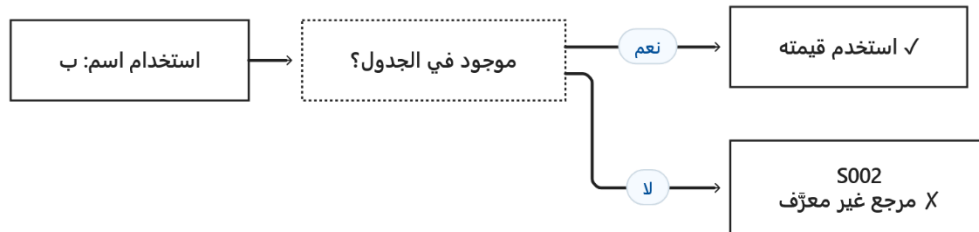
شكل توضيحي — semantic-07

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

#### الفحص 2: مرجع غير معرّف

{ب 5 = ؛ { خطأ: S002: ب "لم يُعرّف مسبقاً

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر



شكل توضيحي — semantic-07

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

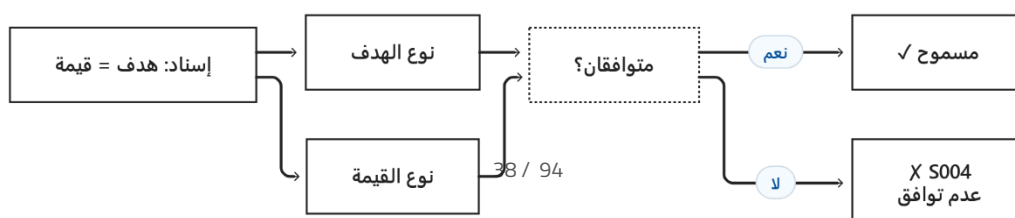
#### الفحص 3: تعديل ثابت

ثابت أقصى 100 = ؛

{أقصى 200 = ؛ { خطأ: S003: لا يمكن تعديل ثابت

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

#### الفحص 4: توافق الأنواع



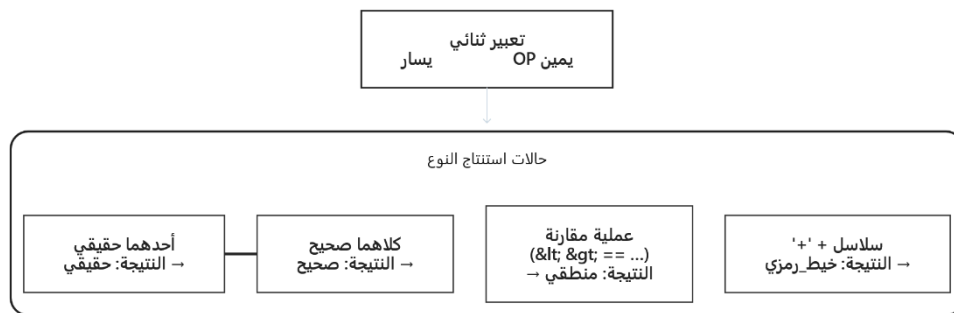
## شكل توضيحي — semantic-07

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية. قواعد التوافق:

صحيح حقيقي ✓ (ترقية ضمنية مسموحة)  
 حقيقي صحيح X (تضييق غير مسموح)  
 أي منطقي ، x لا تحويل ضمني  
 نوع مسقى يُحلّ أولاً ثم يُقارن نوعه الأصلي

## 7.4 استنتاج نوع التعبير

بعد الفحص، يستنتج semantic نوع كل تعبير:



## شكل توضيحي — semantic-07

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 7.5 مسار استدعاء semantic في driver

```
compiler_driver_run()
```

```
[1] yyparse() g_root_ast
```

```
[2] c_analyze_semantics(g_root_ast, &result)
```

تصفح declarations بناء جدول الرموز

تصفح statements فحص كل تعليمة

semantic يستخدم نتيجة tac [3] c\_generate\_()

c\_semantic\_result\_free(&result) [4] بعد بناء الاستجابة

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 7.6 الحدود الموثقة

ما يفعله semantic.c حالياً

✓ جدول رموز خطي (linear) — نطاق واحد مشترك

✓ فحص: تكرار التعريف، مرجع غير معرّف

✓ فحص: تعديل ثابت

✓ فحص: توافق نوع الإسناد

✓ حل الأنواع المسماة

✓ استنتاج نوع التعبيرات

✓ حل وصول عناصر القائمة والسجل

✗ ما لا يفعله حالياً

✗ مكس نطاقات مستقل لكل إجراء (scope stack)

✗ التحقق من عدد ونوع وسائط الإجراء عند الاستدعاء

✗ فرض أن تعبير الشرط من نوع منطقي صراحةً

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 8.1 — التمثيل الوسيط: 3AC و Typed IR

### 8.1.1 فلسفة Three-Address Code

يُفكّك 3AC التعبير المركب إلى تعليمات بسيطة كل منها بثلاثة عناوين أقصى: - نتيجة مؤقتة (\$t1, t0, ...) - معاملان

(قيمتان أو مؤقتتان أو ثوابت) - عامل واحد

الفوائد: - يُقلل تعقيد backend — لا يقرأ AST مباشرة - يتيح التحقق النصي المباشر (golden tests) - يُظهر تدفق

التحكم صريحاً في لوحة 3AC

### 8.1.2 أنواع تعليمات 3AC

| الرمز  | المعنى      | مثال         |
|--------|-------------|--------------|
| ALLOC  | تخصيص متغير | ALLOC س صحيح |
| ASSIGN | إسناد       | س = \$t0     |

|        |               |                   |
|--------|---------------|-------------------|
| BINARY | عملية ثنائية  | \$t0 = أ ب        |
| UNARY  | عملية أحادية  | \$t0 = أ          |
| LABEL  | تسمية         | L0:               |
| JUMP   | قفز غير مشروط | JUMP L2           |
| BRANCH | قفز مشروط     | BRANCH \$t0 L1 L2 |
| CALL   | استدعاء إجراء | CALL إجراء        |
| PARAM  | تمرير معامل   | PARAM أ           |
| READ   | قراءة         | READ س            |
| PRINT  | طباعة         | PRINT \$t0        |

### 8.1.3 مثال: تحويل تعبير إلى 3AC

المصدر: أ = س ب \* 2؛

التحويل:

\$t0 = ب \* 2

\$t1 = س

أ = \$t1

يعالج المولد التعبير من الداخل إلى الخارج: 1. يولد للتعبيرات الفرعية أولاً 2. يسند ناتج كل خطوة لـ 3 temporary. يسند النتيجة النهائية للمتغير الهدف

### 8.1.4 تمثيل التحكم في 3AC

جملة if

BRANCH شرط L\_then L\_else

```

LABEL L_then:
...
JUMP L_end
LABEL L_else:
...
LABEL L_end:

```

حلقة while

```

LABEL L_start:

```

BRANCH شرط L\_body L\_end

```

LABEL L_body:
...
JUMP L_start
LABEL L_end:

```

حلقة repeat-until

```

LABEL L_start:

```

...جسم الحلقة...

BRANCH شرط L\_end L\_start ينفذ الجسم أولاً ثم يفحص

```

LABEL L_end:

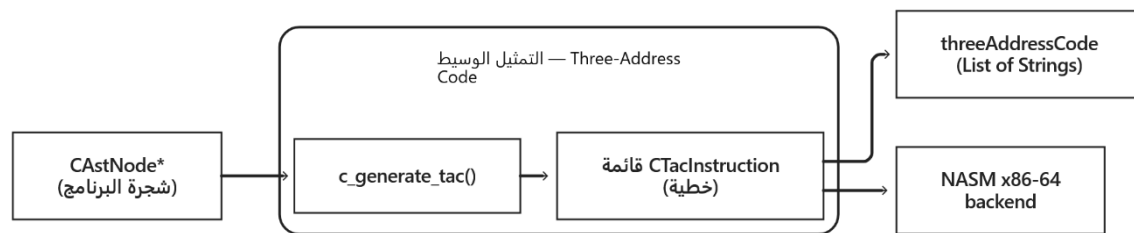
```

## مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 8.1.5 مخطط تدفق التحويل

شكل توضيحي — ir-08

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية. نفس قائمة 3AC التي تُعرض في البروتوكول هي ما يستهلكه NASM backend.



### 8.1.6 Typed IR (ملخص بروتوكول JSON)

الملف: packages/compiler\_c/src/ir/typed\_ir.c

ينتج typed\_ir.c حقل intermediateRepresentation كملخص JSON:

```

{
  "unit": "main",
  "name": "اسم البرنامج",
  "target": "dart-native",
  "types": ["صحيح", "حقيقي", "منطقي"],
  "symbols": [
    { "name": "i", "type": "i", "offset": 8 },
    { "name": "b", "type": "b", "offset": 16 }
  ],
  "blocks": [
    { "name": "main", "statementCount": 5 }
  ]
}
  
```

ما يمثله وما لا يمثله

| لا يمثله                  | يمثله                           |
|---------------------------|---------------------------------|
| تعليمات typed لكل عملية   | ملخص أنواع البرنامج ورموزه      |
| مخطط كتل تحكم (CFG)       | إزاحات الرموز (حساب تسلسلي 8*i) |
| صيغة SSA أو LLVM IR       | عدد التعليمات في كتلة main      |
| مسار مستقل لتوليد الشيفرة | اسم الوحدة والهدف               |

العبرة الحقيقة: البروتوكول يوفر ملخص Typed IR لمعلومات البرنامج. المترجم يولد TAC فعلياً للشفرة الأصلية.

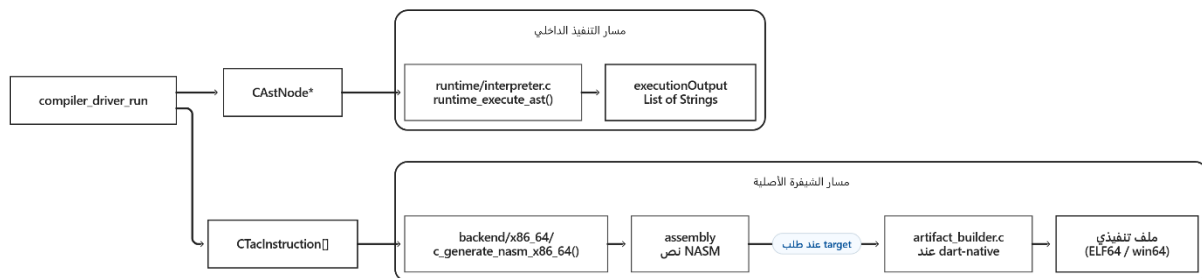
### 8.1.7 ما يثبتته الاختبار الذهبي لـ TAC

اختبار `tac_golden_test.c`: يُثبت بقاء عملية BINARY عند فحص 1 2 (لا طي ثوابت) - لا يظهر في مسار `driver` تمرير مستقل لتحسين TAC يُوصف المترجم بأنه بدون `optimizer` حتى تضاف مرحلة مثبتة بجانب اختباراتها.

## 9.1 — التنفيذ والخلفية وبناء الأثر

### 9.1.1 مسارا التنفيذ

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية. مهم: مسار `interpreter` يقرأ AST مباشرة ولا يستهلك TAC. مسار `NASM` يستهلك TAC ولا يقرأ AST.



شكل توضيحي — backend-09

### 9.1.2 المفسر الداخلي (Interpreter)

الملف: `packages/compiler_c/src/runtime/interpreter.c`

```
runtime_begin(
    //تهيئة القيم والمدخل
    runtime_execute_ast(
        response, root
    )
    //تنفيذ البرنامج
```

ما يدعمه حالياً

| الدعم                    | التعليمة       |
|--------------------------|----------------|
| ✓                        | إسناد (=)      |
| ✓                        | قراءة (اقرأ)   |
| ✓                        | طباعة (اطبع)   |
| ✓                        | شرط (إذا/وإلا) |
| ✓ (مع حارس أقصى للتكرار) | حلقة (طالما)   |

|                                    |                      |
|------------------------------------|----------------------|
| ✓                                  | حلقة مدى (كرر)       |
| X (غير موجود في execute_statement) | استدعاء إجراء (CALL) |
| X (غير موجود في execute_statement) | اعد...حتى            |

### قيود مهمة

- مخزن محدود: 128 متغيراً فقط - executionOutput منفصل عن artifact — نجاح التنفيذ الداخلي ≠ ملف native - لا يستهلك TAC

## 9.1.3 مولد NASM x86-64

الملف: packages/compiler\_c/src/backend/x86\_64/asm\_x86\_64.c

Assembly نص c\_generate\_nasm\_x86\_64(tac, semantic, result)

ما يولده حالياً

| النوع              | الدعم                                                                 |
|--------------------|-----------------------------------------------------------------------|
| الحساب الصحيح      | ✓, -, *, /, %                                                         |
| الحساب الحقيقي     | ✓ (FPU x87)                                                           |
| المنطقية والمقارنة | ✓                                                                     |
| العوامل الأحادية   | ✓                                                                     |
| الإسناد            | ✓                                                                     |
| الطباعة والقراءة   | ✓ (استدعاءات مكتبة C)                                                 |
| استدعاء الإجراء    | X — يُصدر تشخيص: «تعليلة استدعاء الإجراء لا تملك ABI في NASM backend» |

### ملاحظة مهمة

Assembly = نص NASM للفحص الأكاديمي

≠ ملف تنفيذي

لا يصبح artifact حقيقياً إلا بعد تمريره لـ NASM وGCC.

## 9.1.4 بناء الأثر (artifact\_builder)

الملف: toolchain.cg packages/compiler\_c/src/backend/artifact\_builder.c

الشروط اللازمة لبناء artifact

target = "dart-native"

artifactDirectory AND معطى

AND artifactDirectory < rootPath سياسة الأمان

PATH AND متوفر في NASM  
PATH AND متوفر في GCC

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر  
مسار البناء

```
Assembly .asm
↓ NASM
كائن o. (ELF64 على win64, Linux على Windows)
↓ GCC --link
ملف تنفيذي
↓ يُسجّل المسار في artifacts[]
```

الاحترازات التشغيلية

| الاحتياط          | التفصيل                                     |
|-------------------|---------------------------------------------|
| قيد المسار        | artifactDirectory يجب أن يكون داخل rootPath |
| فشل الأداة        | يُضاف إلى diagnostics ويجعل success=false   |
| مهلة العملية      | ProcessCompilerRepository يفرض timeout      |
| إدخال تفاعلي ملغى | يُعيد خطأ runtime واضحاً لا قيمة صفر        |

9.1.5 مثال على Assembly الناتجة

; ناتج من: 2 = 3 : اطبع(أ)؛

```
section .data
    fmt_int db "%d", 10, 0

section .text
    global main
    extern printf

main:
    push rbp
    mov rbp, rsp

    ; $t0 = 2  3
    mov eax, 2
    add eax, 3
    mov [rbp-8], eax    ; t0$ = أ

; اطبع(أ) ;
    mov edi, fmt_int
    mov esi, [rbp-8]
    call printf

    pop rbp
    ret
```

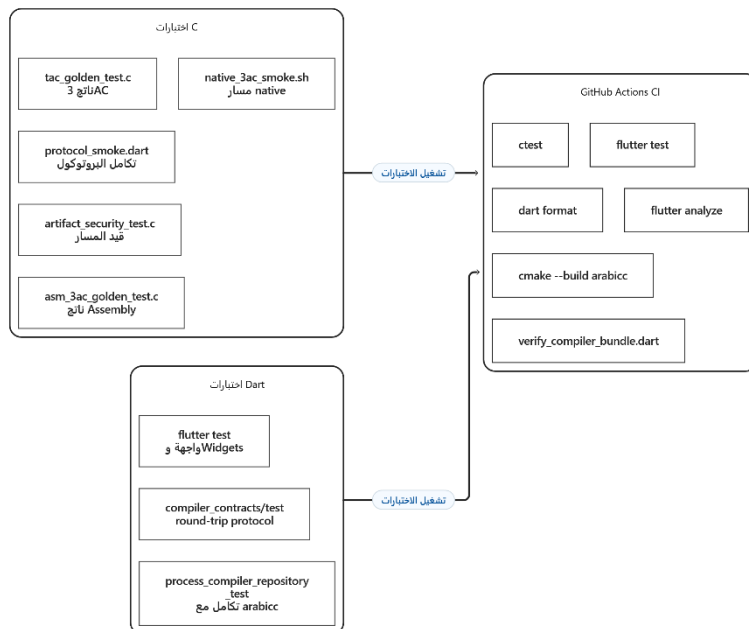
هذا مثال توضيحي — الناتج الفعلي يعتمد على التنفيذ الحالي في 64.c.\_asm\_x86

9.1.6 ملخص الحدود الأساسية

| الادعاء                       | الدليل المتاح     | ما لا يثبتته             |
|-------------------------------|-------------------|--------------------------|
| ASTg Parserg Lexer موجودة     | ملفات Flex/Bison  | اكتمال كل صياغة          |
| Semantic يفحص رموزاً وأنواعاً | semantic.c        | نظام نطاقات كامل         |
| TAC يُولّد ويُعرض             | tac.c golden test | وجود تحسين               |
| NASM يُولّد                   | golden test smoke | دعم كل ABI               |
| Runtime ينفذ AST              | interpreter.c     | تطابقه الكامل مع backend |

## 10.1 — الاختبارات وضمان الجودة والبناء المستمر

### 10.1.1 استراتيجية الاختبار



شكل توضيحي — testing-10

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

### 10.1.2 اختبارات Smoke Golden

| الاختبار                    | ما يتحقق منه                              |
|-----------------------------|-------------------------------------------|
| tac_golden_test.c           | النص الوسيط 3AC مطابق للمتوقع             |
| asm_3ac_golden_test.c       | ناتج Assembly من TAC مطابق                |
| native_3ac_smoke.sh         | مسار native ينتج ملفاً                    |
| protocol_smoke.dart         | إرسال واستقبال JSON واستجابة صحيحة البنية |
| artifact_security_test.c    | منع الكتابة خارج rootPath                 |
| verify_compiler_bundle.dart | arabicc في bundle يرد برد JSON صالح       |

### 10.1.3 طبقات الاختبار وما تثبته

| الطبقة               | المسؤولية                      | لا تثبت            |
|----------------------|--------------------------------|--------------------|
| Dart contract tests  | نماذج JSON والتحقق منها        | تنفيذ compiler     |
| Dart repo tests      | تشغيل arabicc واستقبال استجابة | صحة مراحل compiler |
| C golden tests       | ناتج مرحلة محددة لمثال محدد    | اكتمال اللغة       |
| Flutter widget tests | سلوك الواجهة                   | compiler backend   |
| CTest                | بناء وتشغيل كل اختبارات C      | استهداف كل منصة    |

قاعدة: لا يُغني اختبار الواجهة عن اختبار compiler, ولا العكس.

### 10.1.4 الأمن والحدود

| الاحتراز       | التفصيل                                                     |
|----------------|-------------------------------------------------------------|
| قيد المسار     | artifact_security_test.c يتحقق من منع الكتابة خارج rootPath |
| stability test | تكرار الطلبات دون انهيار                                    |
| قاعدة الإعلان  | لا تُدعى ميزة بدون تنفيذ اختبار دليل تشغيل                  |

### 10.1.5 البناء المستمر (CI)

مسارات ci.yml

```
jobs:
  compiler-c:
    - cmake -S packages/compiler_c -B build
    - cmake --build build
    - ctest --test-dir build

  flutter-app:
    - flutter pub get
    - dart format --check
    - flutter analyze
    - flutter test

  bundle-smoke:
    - dart run tool/verify_compiler_bundle.dart
```

ملاحظات البناء

| المنصة  | الخصوصية                                       |
|---------|------------------------------------------------|
| Linux   | يُبنى CI الرئيسي                               |
| Windows | Flex بوضع wincompat-- (يتجنب unistd.h مع MSVC) |
| macOS   | مدعوم في release workflow                      |

سياسة الإصدار: لا تُضمَّن arabicc في حزمة Desktop إلا بعد اجتياز smoke test.

## 10.1.6 خريطة ملفات المصدر الكاملة

```
Flutter دخول نقطة          lib/main.dart
lib/config/di/injection.dart      ServiceLocator
lib/features/editor/
  presentation/controllers/editor_controller.dart
  presentation/ui/screens/editor_screen.dart
  data/repositories_impl/
    local_workspace_repository_impl.dart
    process_compiler_repository_impl.dart
packages/compiler_contracts/
نماذج JSON                      /lib/src
  test/compilation_protocol_test.dart
packages/compiler_c/
  src/main.c                      CLI protocol entry
src/driver/compiler_driver.c      pipeline
  src/lexer.l                     Flex scanner
  src/parser.y                    Bison parser
  src/ast.c include/ast.h         AST
التحليل الدلالي                src/semantic.c
توليد 3AC                      src/ir/tac.c
  src/ir/typed_ir.c              Typed IR summary
التنفيذ الداخلي              src/runtime/interpreter.c
  src/backend/x86_64/asm_x86_64.c NASM x86-64
بناء artifact                  src/backend/artifact_builder.c
تكمال NASM/GCC                 src/backend/toolchain.c
  tests/tac_golden_test.c
  tests/asm_3ac_golden_test.c
  tests/artifact_security_test.c
  tests/protocol_smoke.dart
tool/native_3ac_smoke.sh
tool/verify_compiler_bundle.dart
.github/workflows/ci.yml
```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## المراجع والمصادر المعتمدة — الفصل الأول

### المراجع الداخلية (المستودع)

| الغرض                                     | الملف                                  |
|-------------------------------------------|----------------------------------------|
| نقطة الحقيقة المعرفية للمشروع             | README.md                              |
| الطبقات وعقد التكامل                      | docs/architecture/architecture.md      |
| شجرة مفصلة، تدفق البيانات، وظائف المكونات | docs/architecture/project-code-map.md  |
| عقد JSON وسلوك البروتوكول                 | docs/architecture/compiler-protocol.md |

|                                      |                                                   |
|--------------------------------------|---------------------------------------------------|
| تنفيذ C compiler وأدلة اختبار وحدوده | docs/architecture/compiler-c-acceptance-matrix.md |
| حدود المنتج والحالة الحالية          | docs/architecture/product-boundary.md             |
| المتطلبات ومخرجات التكليف            | docs/assignment/01-requirements.md                |
| المعمارية والتنظيم الحالي            | docs/assignment/02-architecture.md                |
| قواعد اللغة ومراحل المترجم           | docs/assignment/03-language-and-compiler.md       |
| دليل مكونات الشيفرة                  | docs/assignment/12-code-component-guide.md        |
| استراتيجية الاختبار                  | docs/testing/test-strategy.md                     |
| إعداد البناء                         | packages/compiler_c/CMakeLists.txt                |
| جميع اختبارات C                      | packages/compiler_c/tests/                        |
| بوابة CI والإصدار                    | .github/workflows/ci.yml                          |

## المراجع الأكاديمية

1. Watson, Des. \*A Practical Approach to Compiler Construction\*. Springer, 2017. DOI: 10.1007/978-3-319-52789-5
2. LLVM Project. \*Writing an LLVM Backend.\* <https://llvm.org/docs/WritingAnLLVMBackend.html>
3. Flex Manual — Matching. <https://westes.github.io/flex/manual/Matching.html>
4. Flex Manual — Patterns. <https://westes.github.io/flex/manual/Patterns.html>
5. GNU Bison Manual — Specifying Operator Precedence. [https://www.gnu.org/software/bison/manual/html\\_node/Precedence.html](https://www.gnu.org/software/bison/manual/html_node/Precedence.html)
6. GNU Bison Manual — Error Recovery. [https://www.gnu.org/software/bison/manual/html\\_node/Error-Recovery.html](https://www.gnu.org/software/bison/manual/html_node/Error-Recovery.html)
7. LLVM Language Reference Manual (للمقارنة فقط). <https://llvm.org/docs/LangRef.html>
8. NASM Manual — Output Formats. <https://www.nasm.us/doc/nasm09.html>
9. GCC Manual — Options for Linking. <https://gcc.gnu.org/onlinedocs/gcc/Link-Options.html>

استخدام LLVM و Flex و Bison و NASM و GCC هنا للمقارنة وتوثيق الأدوات فقط. لا يعني ذلك أن المشروع يستخدم LLVM أو أن واجهات مترجمه مطابقة لها.

## الفصل الثاني — البنية الداخلية للمترجم

التقرير الأكاديمي النهائي > مقرر: بناء المترجمات وتصميم لغات البرمجة (Compilers Construction & Programming Languages) > CS & IT4 — (Design) إشراف: د. خالد الكحسة · م. عقيل الشرفي > الطلاب: عمران غالب محمد ناصر · محمد أحمد حميد اليفرسي · أسامة النشمي > التاريخ: 27 سبتمبر 2026

### هيكل الفصل

chapter2/

|                                                  |                         |
|--------------------------------------------------|-------------------------|
| هذا الملف (فهرس الفصل)                           | README.md               |
| الصورة العامة ومسار التنفيذ                      | pipeline.md-01          |
| التحليل المعجمي: Flex والرموز والكلمات المحجوزة  | lexical-analysis.md-02  |
| التحليل النحوي: Bison وبناء AST والقواعد الكاملة | syntax-analysis.md-03   |
| التحليل الدلالي: جدول الرموز والأنواع والفحوص    | semantic-analysis.md-04 |
| التمثيلات الوسيطة: 3AC Typed IR                  | intermediate-repr.md-05 |
| توليد الشيفرة: NASM x86-64 وبناء الأثر           | code-generation.md-06   |
| التنفيذ الداخلي والمفسر والبروتوكول              | runtime.md-07           |
| اختبارات التحقق ومصفوفة القبول وحدود الدليل      | verification.md-08      |
| الخلاصة الهندسية والتوصيات                       | summary.md-09           |
| مرجع اللغة المنفذة (EBNF كامل) أسبقية العوامل    | language-spec.md        |
| المراجع الخارجية                                 | references.md           |

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### خريطة المحتوى

| المفهوم القياسي             | الموضوع                      | الملف                   |
|-----------------------------|------------------------------|-------------------------|
| Compiler Pipeline           | الصورة الكلية والمسارات      | 01-pipeline.md          |
| Lexical Analysis / Scanning | المسح المعجمي — Flex         | 02-lexical-analysis.md  |
| Syntax Analysis / Parsing   | التحليل النحوي — AST         | 03-syntax-analysis.md   |
| Semantic Analysis           | التحليل الدلالي وجدول الرموز | 04-semantic-analysis.md |
| Intermediate Representation | التمثيل الوسيط 3AC Typed IR  | 05-intermediate-repr.md |
| Code Generation Linking     | توليد NASM وبناء ملف تنفيذي  | 06-code-generation.md   |
| Interpreter Runtime         | مفسر AST والبروتوكول         | 07-runtime.md           |
| Verification & Testing      | الاختبارات والأدلة والحدود   | 08-verification.md      |
| Summary                     | الخلاصة والتوصيات            | 09-summary.md           |
| Language Specification      | مرجع EBNF والأسبقية          | language-spec.md        |

## المنهجية

يعرض هذا الفصل البنية الداخلية للمترجم وفق التقسيم الدولي المتعارف عليه:



مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

مع التمييز الصريح في كل قسم بين: - المفهوم القياسي كما في الأدبيات الأكاديمية - ما تنفذه الشيفرة الحالية بدليل الاختبارات والكود - حدود التنفيذ الحالي حتى لا توهي الوثيقة بنطاق أقوى مما ينقذه المصدر

قيد التحقق: لم يُبن المترجم ولم تُشغّل اختبارات C في بيئة إعداد هذا التوثيق لغياب CMake/Flex/Bison. يثبت الفصل قراءة المصدر والاختبارات المكتوبة، لا إعادة تشغيل CI.

## 2.1 — الصورة العامة ومسارا التنفيذ

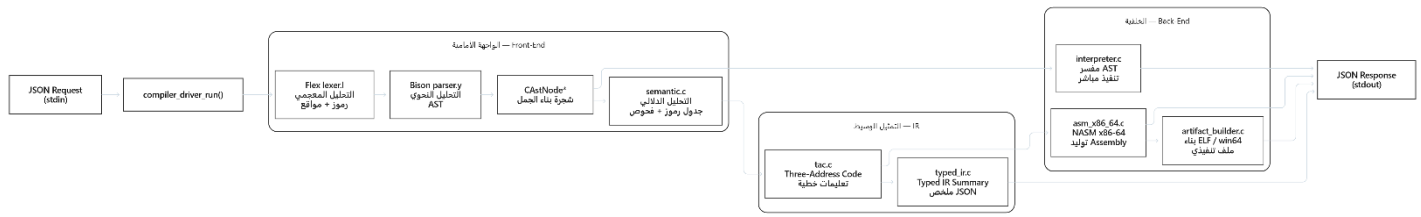
### 2.1.1 تمهيد

يشرح هذا الفصل كيف ينتقل البرنامج العربي من نص مصدر إلى رموز وشجرة نحوية وتمثيلات وسيطة، ثم إلى نص Assembly أو تنفيذ تفسيري.

يعتمد الوصف على التنفيذ الفعلي في `packages/compiler_c` وعلى الاختبارات والتوثيق المجاور في المستودع، لا على أسماء المجلدات أو الخطة النظرية وحدها.

تُعرض البنية وفق التقسيم الشائع إلى: - الواجهة الأمامية — تحليل اللغة (Lexer Parser Semantic) - التمثيل الوسيط — جسر بين المراحل (TAC Typed IR) - الخلفية — توليد الشيفرة (NASM Interpreter Artifact) مع اعتبار التحسين مرحلة ممكنة وليست شرطاً في كل مترجم — ولا تظهر في `driver` الحالي كمرحلة مستقلة.

## 2.1.2 مخطط المسار العام



شكل توضيحي — pipeline-01

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 2.1.3 جدول مراحل المترجم

| # | المرحلة         | المدخل         | الناتج / المسؤولية                     | موضع التنفيذ                     |
|---|-----------------|----------------|----------------------------------------|----------------------------------|
| 1 | نقل الطلب       | JSON عبر stdin | مصدر خيارات حقول البروتوكول            | compiler_driver.cg /src/protocol |
| 2 | التحليل المعجمي | محارف المصدر   | رموز نصوصها مواقعها                    | src/lexer.l (Flex)               |
| 3 | التحليل النحوي  | الرموز         | عقد AST تشخيصات النحو                  | src/parser.y (Bison)             |
| 4 | التحليل الدلالي | AST            | قائمة رموز فحوص التعريفات فحوص الأنواع | src/semantic.c                   |
| 5 | 3AC             | AST            | تعليمات TAC خطية                       | src/ir/tac.c                     |
| 6 | Typed IR        | AST Semantic   | ملخص JSON للأنواع والرموز              | src/ir/typed_ir.c                |
| 7 | توليد الشيفرة   | TAC            | نص x86-64 NASM                         | src/backend/x86_64/asm_x86_64.c  |
| 8 | إنشاء الأثر     | Assembly محفوظ | كائن ELF/win64 ثم ملف تنفيذي           | toolchain.cg artifact_builder.c  |
| 9 | التنفيذ الداخلي | AST مدخلات     | قيم مخرجات التنفيذ                     | src/runtime/interpreter.c        |

## 2.1.4 المسار ليس سلسلة خطية واحدة

هذه نقطة معمارية جوهرية يجب استيعابها:

AST ► interpreter.c

## يقرأ AST مباشرة

▼  
TAC

► asm\_x86\_64.c

## يستهلك تعليمات TAC

## (عند dart-native artifactDirectory ▼)

Assembly

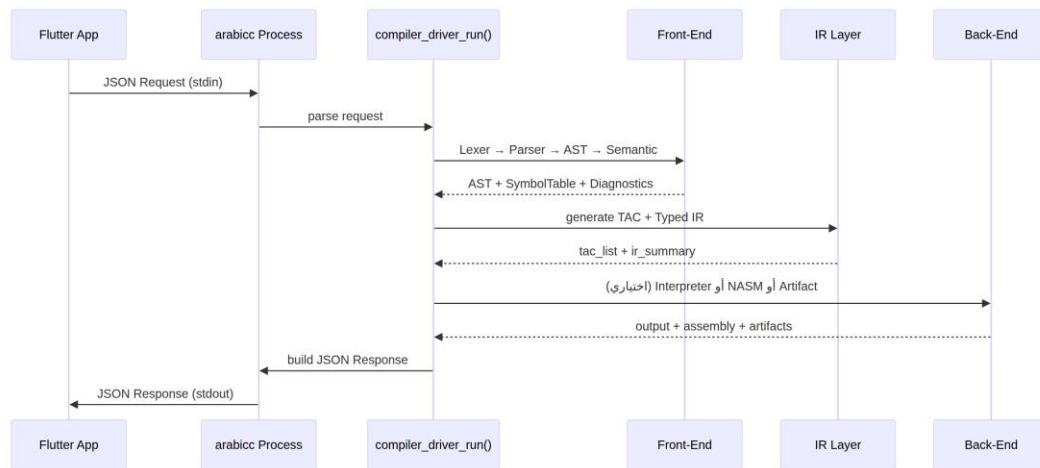
► artifact\_builder.c

NASM GCC

ملف تنفيذي ELF/win64

| النقطة                | التفصيل                                                      |
|-----------------------|--------------------------------------------------------------|
| مفسر التشغيل          | يقرأ AST مباشرة — لا يستهلك TAC ولا Assembly                 |
| مولد NASM             | يستهلك TAC — لا يقرأ AST بعد هذه المرحلة                     |
| assembly في الاستجابة | نص وسيط — لا يصح artifact إلا بعد تشغيل NASM GCC             |
| لا مرحلة تحسين مستقلة | لم تظهر في ترتيب compiler_driver_run الحالي بين TAC backendg |
| TypedIR               | يُنْتِج من AST نتيجة Semantic, ليس من TAC                    |

## 2.1.5 دورة الطلب والاستجابة



شكل توضيحي — pipeline-01

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

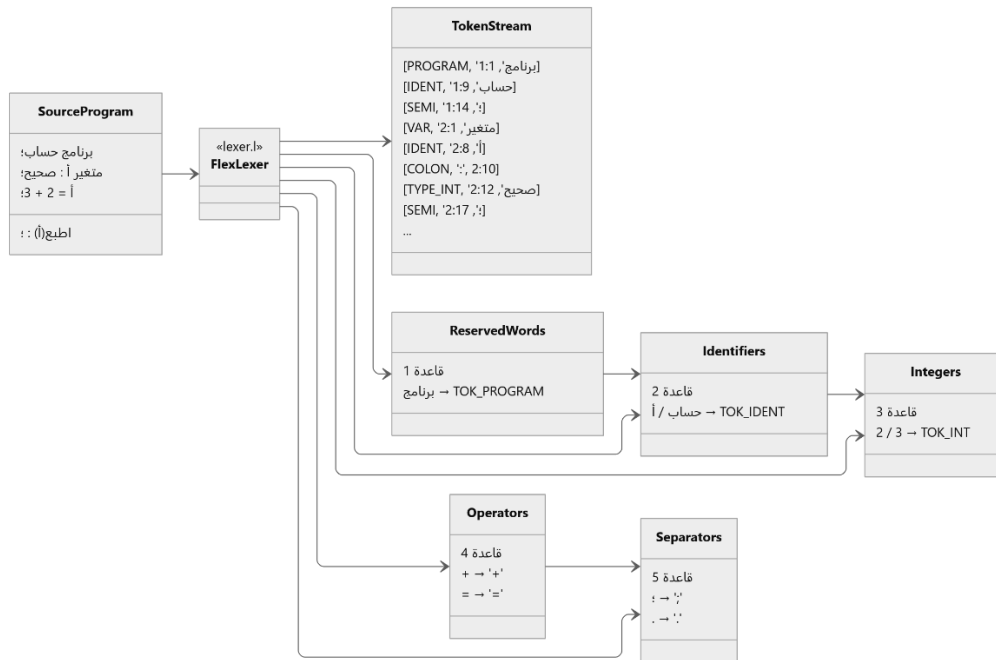
ملاحظة: قد تعمل الاستجابة على أكثر من مرحلة حتى إن وجدت تشخيصات أخطاء — لذلك ينبغي قراءة حقل كل مرحلة على حدة، لا الاستنتاج من قيمة success وحدها.

## 2.2 — التحليل المعجمي (Lexical Analysis)

التحليل المعجمي هو أول مرحلة في المترجم. مهمته تحويل النص العربي الخام — سلسلة من المحارف — إلى قائمة منظمة من الرموز (Tokens) يستطيع المحلل النحوي التعامل معها.

### 2.2.1 ما يفعله Lexer بكلمات بسيطة

تخيّل أنك تقرأ جملة عربية — أول شيء يفعله دماغك أنه يُقسّم النص إلى كلمات ورموز قبل أن يفهم معناها. هذا بالضبط ما يفعله Lexer: يقرأ النص حرفاً بحرف ويقول: "هذه كلمة محبوزة، هذا معرّف، هذا رقم، هذا عامل".



شكل توضيحي — lexical-analysis-02

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكوّن ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

### 2.2.2 آلية عمل Flex

يستخدم المترجم Flex (Fast Lexical Analyzer Generator) لتوليد المحلل المعجمي تلقائياً من ملف القواعد lexer.إ.

مبدأ أطول تطابق (Maximal Munch)

قاعدة Flex: دائماً اختر الرمز الأطول تطابقاً

مثال:

نص المصدر: "طالما"

تطابق 1: "طا" ليس كلمة محبوزة

تطابق 2: "طالما" TOK\_WHILE ✓ (أطول)

نص المصدر: "برنامج2"

تطابق 1: "برنامج" TOK\_PROGRAM

تطابق 2: "برنامج2" TOK\_IDENT ✓ (أطول — يكسب المعرّف)

### أولوية القاعدة عند التساوي

lexer.a — ترتيب القواعد مهم:

1. كلمات محجوزة (برنامج، ثابت، اذا...) أولاً دائماً
2. معرّفات (أي اسم عربي/ASCII/\_)
3. أعداد حقيقية (3.14 — يجب قبل الصحيحة)
4. أعداد صحيحة (42)
5. سلاسل نصية ("مرحباً")
6. محارف (أ')
7. عوامل وفواصل (، ، - ، ؛ ، ...)
8. تعليقات (/\* ... \*/) لا تُرجع رمزاً لـ Bison
9. رمز مجهول L001 تشخيص

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 2.2.3 الكلمات المحجوزة وأصناف الرموز

جدول الكلمات المحجوزة الكامل

| الصفة الوظيفية | الألفاظ المقبولة حرفياً في lexer.a | رمز Bison     |
|----------------|------------------------------------|---------------|
| بدء البرنامج   | برنامج                             | TOK_PROGRAM   |
| تعريف ثابت     | ثابت                               | TOK_CONST     |
| تعريف نوع      | نوع                                | TOK_TYPE      |
| تعريف متغير    | متغير                              | TOK_VAR       |
| إجراء          | اجراء، إجراء                       | TOK_PROCEDURE |
| تمرير بالقيمة  | بالقيمة                            | TOK_BY_VALUE  |
| تمرير بالمرجع  | بالمرجع                            | TOK_BY_REF    |
| طباعة          | اطبع                               | TOK_PRINT     |
| قراءة          | اقرأ، اقرأ                         | TOK_READ      |
| شرط — رأس      | إذا، إذا                           | TOK_IF        |
| شرط — ثم       | فان                                | TOK_THEN      |
| شرط — وإلا     | والا، وإلا                         | TOK_ELSE      |
| حلقة — شرط     | طالما                              | TOK_WHILE     |
| حلقة — استمر   | استمر                              | TOK_DO        |
| حلقة — عدّية   | كرر                                | TOK_REPEAT    |
| حلقة — من      | من                                 | TOK_FROM      |
| حلقة — إلى     | الى                                | TOK_TO        |
| حلقة — خطوة    | اضف، أضف                           | TOK_STEP      |
| حلقة — حتى     | حتى                                | TOK_UNTIL     |

|                 |          |            |
|-----------------|----------|------------|
| TOK_AGAIN       | اعد, أعد | حلقة — أعد |
| TOK_TYPE_INT    | صحيح     | نوع صحيح   |
| TOK_TYPE_REAL   | حقيقي    | نوع حقيقي  |
| TOK_TYPE_BOOL   | منطقي    | نوع منطقي  |
| TOK_TYPE_CHAR   | حرفي     | نوع حرفي   |
| TOK_TYPE_STRING | خيط رمزي | نوع سلسلة  |
| TOK_ARRAY       | قائمة    | مصفوفة     |
| TOK_RECORD      | سجل      | سجل        |
| TOK_TRUE        | صح       | صح         |
| TOK_FALSE       | خطأ      | خطأ        |

ملاحظة الإملاء: ليست كل بدائل الهمزة موجودة. مثلاً يقبل Lexer الى (بلا همزة) ولا يعرّف إلى. كذلك اطبع بلا بديل آخر. قائمة اقتراحات البروتوكول هي واجهة مساعدة مستقلة وليست مصدر قواعد التحليل.

## 2.2.4 تحويل الرموز الخاصة

يُجري Lexer تحويلات شفافة تُؤدّ رموز التشغيل قبل إرسالها إلى Bison:

| الرمز المكتوب                  | التحويل     | السبب               |
|--------------------------------|-------------|---------------------|
| ؛ (فاصلة منقوطة عربية)         | ';' ASCII   | توحيد المعالجة      |
| , (فاصلة عربية)                | ',' ASCII   | توحيد المعالجة      |
| =<                             | TOK_LE      | بديل لـ >=          |
| =>                             | TOK_GE      | بديل لـ <=          |
| =!                             | TOK_NE      | بديل لـ !=          |
| \\ (شرطة عكسية مفردة أو مكررة) | TOK_INT_DIV | عامل القسمة الصحيحة |

## 2.2.5 العوامل والترقيم الكامل

| المجموعة        | الرموز المقبولة  |
|-----------------|------------------|
| المساواة وعدمها | ==, !=, >=, <=   |
| المقارنة        | <, >, <=, >=     |
| المنطقية        | !, &&,           |
| الحسابية        | ^, %, /, *, -, + |
| القسمة الصحيحة  | \\ أو \          |
| الأقواس         | (, ), {, }, [, ] |
| الترقيم         | :, ;, ,, ,       |

## 2.2.6 قواعد المعجم للأنواع المختلفة

### المعرّفات

ARABIC\_CHAR = نطاقات UTF-8 المحددة لأحرف العربية

```
IDENT_START = ARABIC_CHAR | [a-zA-Z_]
IDENT_CONT  = IDENT_START | [0-9]
IDENTIFIER  = IDENT_START IDENT_CONT*
```

أمثلة صحيحة: متغير\_سرعة, حساب2, x\_القيمة

أمثلة خاطئة: 2متغير (يبدأ برقم)

### الأعداد

```
INTEGER = [0-9]
REAL    = [0-9] '.' [0-9]
```

ملاحظة مهمة: لا تحتوي القاعدة العددية على إشارة.  
الإشارة السالبة أو الموجبة تُقرأ كعامل أحادي في parser.

أمثلة صحيحة: 42, 3.14, 0.5

أمثلة خاطئة: 5. (لا نقطة بدون رقم قبلها)

### السلاسل النصية والمحارف

STRING = ""\*(. '\\" | [\\\"^]) "" يقبل escape sequences

CHAR مفرد = "\". \" ASCII

بديل بعلامة اقتباس | \". \"

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### التعليقات

```
COMMENT = ' ' [^\n]*
```

يُضاف التعليق إلى قائمة الرموز المعروضة للمستخدم،  
لكنه لا يُعاد رمزاً نحوياً إلى Bison — يُتجاهل نحوياً.

## 2.2.7 معالجة النص العربي UTF-8

المحرف العربي يشغل 2 إلى 4 بايت في ترميز UTF-8, لذلك يتتبع Lexer موضعين مستقلين:

نص: "برنامج" (7 محارف عربية)

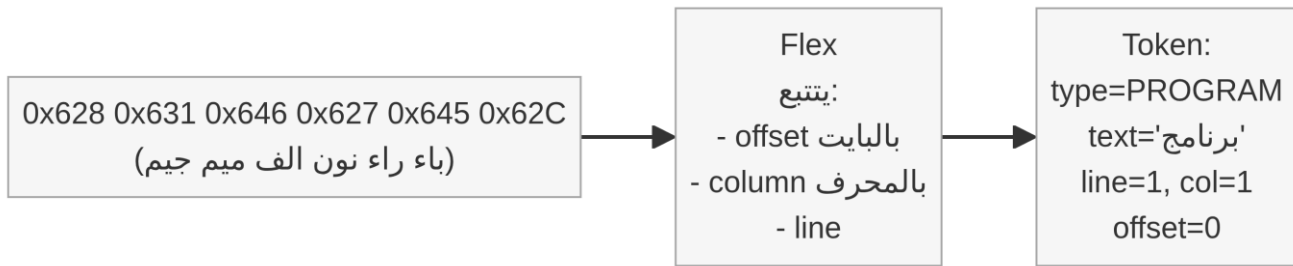
offset: 0 إزاحة بالبايتات من بداية الملف

bytes 14 عند الانتهاء (7 × 2 بايت)

column: 1 رقم العمود بالمحارف (للعرض للمستخدم)

8 عند الانتهاء (7 محارف 1)

line: يُحدَّث عند كل LF



شكل توضيحي — lexical-analysis-02

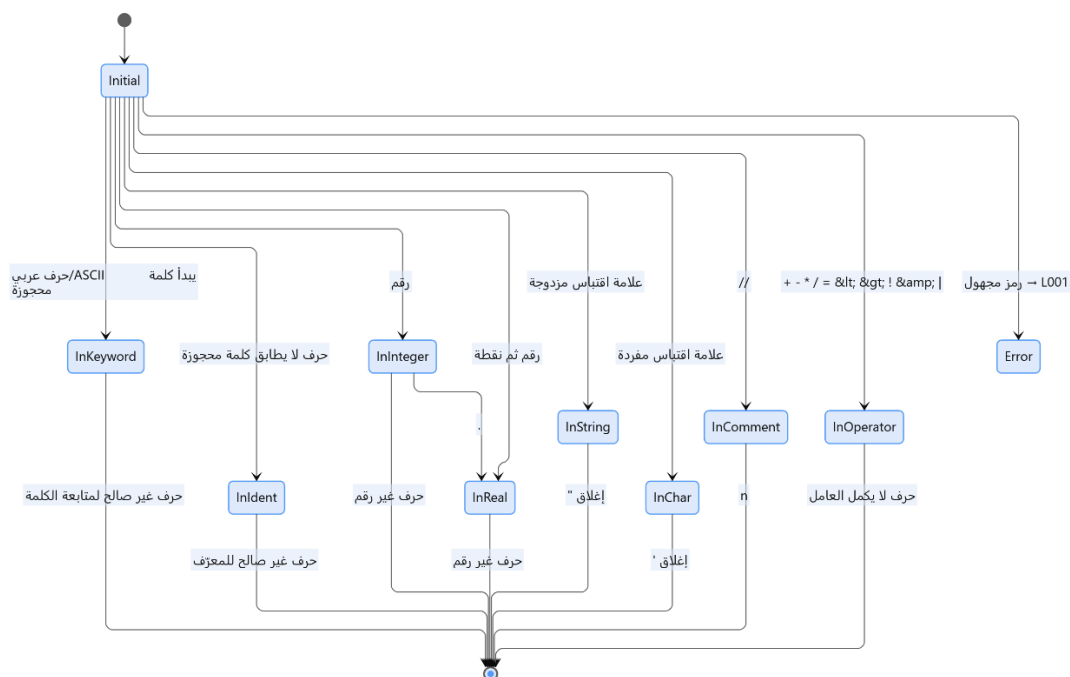
يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 2.2.8 التشخيصات المعجمية

| كود التشخيص | السبب         | مثال                       |
|-------------|---------------|----------------------------|
| L001        | رمز غير معروف | @متغير — الرمز @ غير مقبول |

يصدر تشخيص L001 عند توافر استجابة البروتوكول ويُضَمَّن في حقل diagnostics.

## 2.2.9 مخطط انتقال حالات Lexer المبسط



شكل توضيحي — lexical-analysis-02

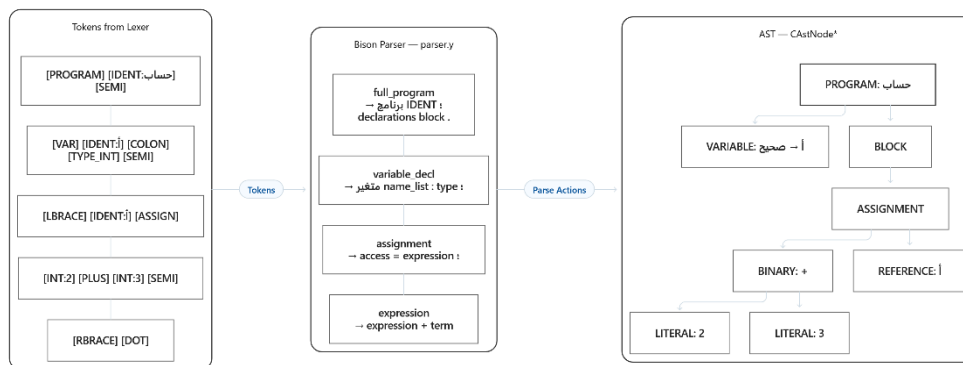
يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 2.3 — التحليل النحوي وبناء AST

بعد أن ينتهي Lexer من تقطيع النص إلى رموز، يأتي Parser ليسأل: هل هذه الرموز مرتبة ترتيباً منطقياً يُشكّل جملاً صحيحة وفق قواعد اللغة؟ وعند قبول الجملة، يبني Parser شجرة (AST) تمثل البنية الهرمية للبرنامج.

### 2.3.1 دور المحلل النحوي

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.



### 2.3.2 نقطة البداية الفعلية

```
program
نقطة البداية الحقيقية الفعلية ▶ full_program
"برنامج" IDENT ";"
{ declaration "
block " }
```

القواعد القديمة مثل (statement, expr) موجودة في الملف لكنها لا تُوصَل من full\_program لأنها لغة مقبولة فعلية

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

تحذير مهم: لا تخط بين القواعد القديمة الموجودة في parser.y والقواعد الفعلية المتصلة بنقطة البداية full\_program. المرجع الحاسم هو القواعد التي يمكن الوصول إليها من program.

### 2.3.3 القواعد النحوية الكاملة (EBNF منظم)

(\* نقطة الدخول \*)

```
" declarations block " ; " IDENT " program ::= "
```

(\* التصريحات \*)

```
" } ; declarations ::= { declaration "
declaration ::= constant_decl
| type_decl
| variable_decl
| procedure_decl ;
```

### (\* أنواع التصريحات \*)

```
" IDENT "=" expression ; ثابتconstant_decl ::= "
" IDENT "=" type ; نوعtype_decl ::= "
" name_list ":" type ; متغيرvariable_decl ::= "
name_list ::= IDENT { "," IDENT } ;
```

### (\* تعريف الإجراء \*)

```
" block ;" IDENT "(" [ parameters ] ")" "اجراءprocedure_decl ::= "
" parameter_group } ; parameters ::= parameter_group { "
" ) name_list ":" type ; بالمرجعparameter_group ::= ( "
```

### (\* الأنواع \*)

```
" type صحيح" | "حقيقي" | "منطقي" | "حرفي" | "خيط رمزي" ::= "
| IDENT
```

### type قائمة " ( [ INTEGER | IDENT ] ) " من "

```
" ] " ;" field } [ "؛" "{" field { سجل" | "
field ::= IDENT ":" type ;
```

### (\* الكتلة والتعليقات \*)

```
block ::= "{" statements "}" ;
" ] } ;" | if_statement [ ":" statements ::= { statement "
statement ::= assignment
| read
| print
| call
| if_statement
| while_statement
| counted_repeat
| repeat_until
| block ;
```

### (\* التعليمات المفردة \*)

```
assignment ::= access "=" expression ;
" "(" access ")" "اقرأread ::= "
" "(" [ arguments ] ")" "اطبعprint ::= "
call ::= IDENT "(" [ arguments ] ")" ;
arguments ::= expression { "," expression } ;
```

### (\* تعليمات التحكم \*)

```
" statement ; استمر" "(" expression ")" "طالماwhile_statement ::= "
" expression "الى" "(" IDENT "=" expression "كررcounted_repeat ::= "
```

### ; statement "(" [ expression ( "أضف" | "أضف" ) ]

```
" "(" expression ")" "حتى" statement "أعد" | "أعدrepeat_until ::= ( "
؛" statement "فان" "(" expression ")" "إذاif_statement ::= "
```

### "|" statement "فان" "(" expression ")" "إذا"

### "|" statement "والا"

### "|" statement "فان" "(" expression ")" "إذا"

### ; if\_statement "والا"

### (\* وصول وتعابير \*)

```
access ::= IDENT { "[" expression "]" | "." IDENT } ;
expression ::= expression ( "|" | "&&" ) expression
| expression ( "==" | "!=" | "<" | ">" | "<=" | ">=" ) expression
| expression ( " " | "-" ) expression
| term ;
term ::= term ( "*" | "/" | "%" | INT_DIV | "^" ) factor
| factor ;
factor ::= INTEGER | REAL | STRING | CHAR
```

### "|" "صح" | "خطأ"

```
| access
| "!" factor | " " factor | "-" factor
| "(" expression ")" ;
```

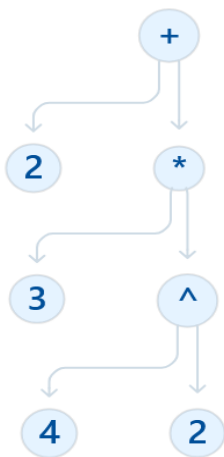
## 2.3.4 أسبقية العوامل وارتباطها

تُعلن الأسبقية في parser.y من الأدنى إلى الأعلى (الأدنى أولاً):

| الترتيب  | العوامل              | الارتباط   | الملاحظة                    | يساري | أو منطقي |
|----------|----------------------|------------|-----------------------------|-------|----------|
| 1 (أدنى) | \                    | \          |                             |       |          |
| 2        | &&                   | يساري      | و منطقي                     |       |          |
| 3        | ==, !=, <, >, <=, >= | يساري      | مقارنات                     |       |          |
| 4        | , -                  | يساري      | جمع وطرح                    |       |          |
| 5        | *, /, %, INT_DIV     | يساري      | ضرب وقسمة                   |       |          |
| 6 (أعلى) | ^                    | يميني      | أس — يرتبط من اليمين        |       |          |
| خاص      | UPLUS, UMINUS, !     | يميني معلن | أحادية — يلزم التحقق بأمثلة |       |          |

تنبيه: UPLUS و UMINUS! مُعلنَة بارتباط يميني، لكن productions العامل الأحادي لا تستخدم UMINUS% prec صراحة في جميع الحالات. لا تستنتج الأسبقية من التصريح وحده — اختبرها بأمثلة.

مثال على حساب الأسبقية



تعبير:  $2^4 * 3 \ 2$

بالأسبقية المُعلنَة يُحلّ كالآتي:

أعلى أسبقية، ارتباط يميني  $16 = 2^4$

ضرب  $48 = 16 * 3$

جمع (أدنى أسبقية)  $50 = 48 \ 2$

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 2.3.5 عقد AST وأنواعها

تُنتج القواعد عقد من النوع CastNode وقوائم CastNodeList:

```

/* أنواع العقد الرئيسية في ast.h */
typedef enum {
/* بنية البرنامج */
    C_AST_PROGRAM,      برنامج اسم تصريحات كتلة
    C_AST_BLOCK,        تعليمات
    /* التصريحات */
    C_AST_CONST_DECL,   ثابت اسم = تعبير
    C_AST_TYPE_DECL,    نوع اسم = نوع
    C_AST_VAR_DECL,     متغير قائمة_أسماء : نوع
    C_AST_PROC_DECL,    إجراء اسم (معاملات) كتلة
} CastNode;

/* التعليمات */

```

```

وصول = تعبير      ,C_AST_ASSIGNMENT
اقرا ( وصول )      ,C_AST_READ
اطبع ( تعبيرات )    ,C_AST_PRINT
اسم ( وسائط )      ,C_AST_CALL
إذا شرط ثم [وإلا] ,C_AST_IF
 طالما شرط جسم    ,C_AST_WHILE
كُرر متغير من إلى [خطوة] جسم ,C_AST_REPEAT
أعد جسم حتى شرط  ,C_AST_REPEAT_UNTIL

/* التعبيرات */
تعبير عامل_ثنائي تعبير ,C_AST_BINARY
عامل_أحادي تعبير      ,C_AST_UNARY
قيمة حرفية (رقم, نص, منطقي, محرف) ,C_AST_LITERAL
مرجع لمعرّف          ,C_AST_REFERENCE

/* الوصول */
مصفوفة [ فهرس ]      ,C_AST_INDEX_ACCESS
سجل . حقل            ,C_AST_FIELD_ACCESS
} CAstNodeKind;

```

## بنية العقدة

```

typedef struct CAstNode {
    نوع العقدة      CAstNodeKind kind
    ;int      offset
    ;int      line
    ;int      col
    union {
/* بيانات مخصصة لكل نوع عقدة */
        struct { char* name; CAstNodeList* decls; struct CAstNode* block; } program;
        struct { struct CAstNode* lhs; struct CAstNode* rhs; } assignment;
        struct { int op; struct CAstNode* left; struct CAstNode* right; } binary;
        struct { int op; struct CAstNode* operand; } unary;
        struct { CTypeSpec type_spec; } literal;
        /* ... */
    } data;
} CAstNode;

```

## 2.3.6 بناء AST خطوة بخطوة

الإدخال:  $2 = 3 * 4$ ؛

خطوة 1: Parser يرى IDENT:'أ'

يتعرف على C\_AST\_REFERENCE access ("أ")

خطوة 2: يرى '='

assignment action يبدأ

خطوة 3: يرى 2:INT

C\_AST\_LITERAL (2, type= صحيح)

خطوة 4: يرى ''

بانتظار الطرف الأيمن

خطوة 5: يرى 3:INT

C\_AST\_LITERAL(3)

خطوة 6: يرى '\*'

أسبقية عالية — يُعالج قبل ''

خطوة 7: يرى 4:INT

C\_AST\_LITERAL(4)

خطوة 8: يبني C\_AST\_BINARY('\*', 3, 4)

خطوة 9: يبني C\_AST\_BINARY(' ', 2, binary(3\*4))

خطوة 10: يبني C\_AST\_ASSIGNMENT (ref(أ), binary)

الشجرة النهائية:

ASSIGNMENT

REFERENCE: أ

BINARY:

LITERAL: 2

BINARY: \*

LITERAL: 3

LITERAL: 4

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 2.3.7 أمثلة على أشجار AST لتعليمات مختلفة

مثال: تعريف متغير وإسناد

مثال: برنامج

صحيح: : س متغير

{ 5 = س }

.

```

PROGRAM: "مثال"
VAR_DECL: "س" TYPE(صحيح)
BLOCK
ASSIGNMENT
REFERENCE: "س"
LITERAL: 5

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

مثال: جملة شرطية

إذا (س < 0) فإن اطبع(س)؛ وإلا اطبع(0)؛

```

IF
(LITERAL(0), REF(س), '<')condition: BINARY
then: PRINT
REF(س)
else: PRINT
LITERAL(0)

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

مثال: حلقة عدّية

كرر (ع = 1 إلى 10 أضع 1) اطبع(ع)؛

```

REPEAT
("ع")var: REFERENCE
from: LITERAL(1)
to: LITERAL(10)
step: LITERAL(1)
[REF(ع)] body: PRINT

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

مثال: تعريف سجل ووصول

```

{ حقيقي : ص ؛ حقيقي : س } سجل = نقطة نوع
نقطة : ن متغير
{ ؛ 3.14 = س.ن }

```

```

" RECORDنقطةTYPE_DECL: "
FIELD: " TYPE(س حقيقي)
FIELD: " TYPE(ص حقيقي)

VAR_DECL: " TYPE_REF("نقطة")

ASSIGNMENT
FIELD_ACCESS
REFERENCE: "ن"
field: "س"
LITERAL: 3.14

```

## مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### 2.3.8 تشخيصات النحو

| كود  | الحالة        | مثال                       |
|------|---------------|----------------------------|
| نحوي | رمز غير متوقع | أ = ؛ — لا تعبير بعد =     |
| نحوي | فاصلة مفقودة  | برنامج مثال متغير أ : صحيح |
| نحوي | إغلاق مفقود   | { أ = 5 بدون }             |

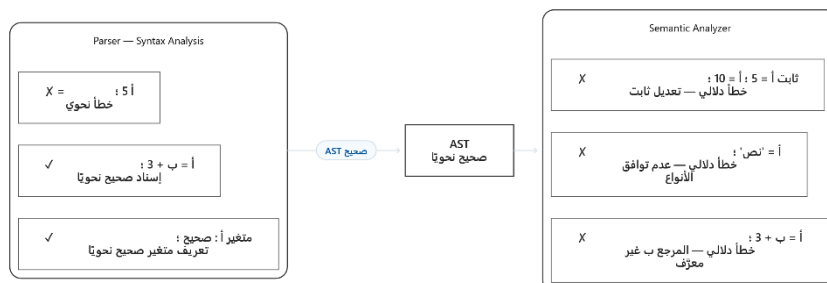
تتضمن عقد AST معلومات الموضع (line, col, offset) التي تُستخدم في رسائل التشخيص لإرشاد المستخدم إلى مكان الخطأ.

### 2.3.9 حدود التنفيذ الحالي

| النقطة                         | الوضع                                                             |
|--------------------------------|-------------------------------------------------------------------|
| طول المصفوفة بمعرف             | مقبول نحوياً، لكن CTypeSpec.length = 0 في action الخاص به         |
| تمرير المعاملات بالقيمة/المرجع | مقبول نحوياً، اكتمال الدعم يتبع المرحلة اللاحقة                   |
| حلقة repeat_until              | مقبولة نحوياً، لكن interpreter.c لا تحتوي حالة C_AST_REPEAT_UNTIL |
| استدعاء الإجراءات              | مقبول نحوياً، NASM backend يُصدر تشخيصاً لغياب ABI                |

## 2.4 — التحليل الدلالي ونموذج الأنواع

التحليل الدلالي هو المرحلة التي تسأل: هل البرنامج منطقي؟ يمكن للجملة أن تكون صحيحة نحوياً تماماً ولكنها بلا معنى — مثلاً تعديل ثابت، أو جمع رقم مع نص غير متوافق. هنا يتدخل semantic.c.



### 2.4.1 الفرق بين النحو والدلالة

### شكل توضيحي — semantic-analysis-04

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

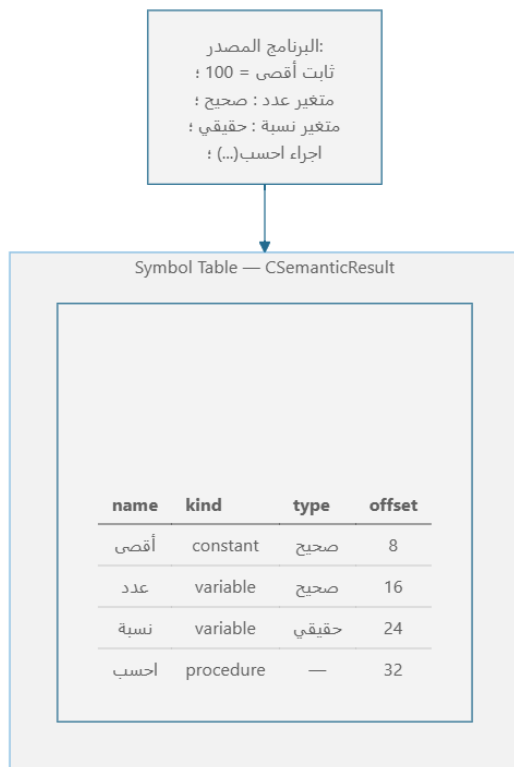
## 2.4.2 جدول الرموز (Symbol Table)

ما هو؟

جدول الرموز هو قاموس البرنامج؛ يسجل كل اسم ظهر في الكود، من أين جاء، ما نوعه، وهل هو ثابت أم متغير أم إجراء.

### شكل توضيحي — semantic-analysis-04

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج، وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.



## بنية السجل في C

```
/* بنية سجل رمز واحد */
typedef struct {
    char* name;          // اسم الرمز (UTF-8)
    CTypeKind type_kind; // صحيح | حقيقي | منطقي | حرفي | خيط_رمزي | سجل | قائمة | غير معروف
    CSymKind sym_kind;   // variable | constant | procedure | type
    int offset;          // إزاحة في جدول الرموز: 8 * index
    SourceSpan span;     // موضع التعريف (line, col, offset)
} SymbolRecord;

/* نتيجة التحليل الدلالي كاملاً */
typedef struct {
```

مصفوفة السجلات ;SymbolRecord\* symbols

```
int symbol_count;
```

قائمة الأخطاء والتحذيرات ;Diagnostic\* diagnostics

```
int diag_count;
} CSemanticResult;
```

## حساب الإزاحة

$8 \times \text{index} = \text{offset}$  (index في الجدول)  $\times 8$

أقصى index 0 offset =  $0 \times 8 = 8$

عدد index 1 offset =  $1 \times 8 = 16$

نسبة index 2 offset =  $2 \times 8 = 24$

احسب index 3 offset =  $3 \times 8 = 32$

الإزاحة هي حساب تسلسلي من ترتيب الرموز، وليست إزاحة عنوان ذاكرة مضمونة في كل مسارات التنفيذ.

## 2.4.3 الفحوص التي يُجريها semantic.c

### الفحص 1: التعريف المكرر

متغير أ : صحيح ؛

متغير أ : حقيقي ؛ خطأ S001: الاسم "أ" معرّف مسبقاً

يفحص semantic.c الجدول قبل إضافة أي رمز جديد.

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### الفحص 2: المرجع غير المعرّف

؛ صحيح : أ متغير

معرّف غير "ب": S002: خطأ { 1 : أ = ب }

يُستخدم معرّف كل عن الجدول في بحث

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### الفحص 3: الإسناد إلى ثابت

؛ أقصى ثابت = 100

"أقصى" الثابت تعديل يمكن لا: S003: خطأ { 200 : أقصى = }

الإسناد قبول قبل sym\_kind يفحص

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

### الفحص 4: توافق الأنواع في الإسناد

؛ صحيح : أ متغير

"صحيح" مع يتوافق لا "رمزي\_خط" نوع: S004: خطأ { ؛ "نص" = أ }

الحالية التوافق قاعدة

صحيح ✓

(مسموحة تلقائية ترقية) صحيح حقيقي ✓

حقيقي حقيقي ✓

(متوافق غير) رمزي\_خط صحيح X

#### 2.4.4 استنتاج أنواع التعبيرات

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

## 2.4.5 تفريد الأنواع المسماة (Type Resolution)

1. "يجد" ن "في الجدول" kind = variable, type = TYPE\_REF نقطة")
2. "يبحث عن" نقطة(...) kind = type, type = RECORD
3. "يجد الحقل" س type = "حقيقي حقيقي"
4. "يقارن مع" "1.5" حقيقي، حقيقي ✓

### 2.4.6 حدود التنفيذ الحالي

| الجانب              | ما يُنفَّذ                       | ما لا يُنفَّذ                                                |
|---------------------|----------------------------------|--------------------------------------------------------------|
| جدول الرموز         | قائمة خطية مشتركة لجميع النطاقات | مكدس بيانات نطاق منفصلة لكل إجراء                            |
| فحص استدعاء الإجراء | يراجع تعبيرات الوسائط            | لا يتحقق من أن المسمى إجراء معرّف أو من عدد الوسائط وأنواعها |
| شرط إذا وطالما      | يُحلّل شرط التعبير               | لا يفرض أن النوع منطقي صراحة                                 |

|                    |   |                                  |
|--------------------|---|----------------------------------|
| النطاقات المتداخلة | — | لا يوجد نطاق مستقل داخل كل إجراء |
|--------------------|---|----------------------------------|

## 2.4.7 مخطط تدفق التحليل الدلالي

## 2.5 — التمثيلات الوسيطة

بعد التحليل الدلالي، تنتقل البيانات إلى طبقة التمثيل الوسيط. تقع هذه الطبقة بين الواجهة الأمامية (التي تفهم اللغة) والخلفية (التي تولّد الشيفرة). وظيفتها: تحويل الشجرة الهرمية (AST) إلى تعليمات خطية أبسط يستطيع المولد التعامل معها مباشرة.

في المترجم الحالي يوجد نوعان من التمثيل الوسيط لهما دوران مختلفان جذرياً:

| التمثيل                        | الملف             | الدور الحقيقي                             |
|--------------------------------|-------------------|-------------------------------------------|
| Three-Address Code (3AC / TAC) | src/ir/tac.c      | تعليمات خطية تُغذي مولد NASM مباشرة       |
| Typed IR Summary               | src/ir/typed_ir.c | ملخص JSON للبروتوكول — ليس stream تعليمات |

### 2.5.1 Three-Address Code (3AC)

#### المفهوم العام

الـ 3AC هو شكل وسيط تكون فيه كل تعليمة عملية واحدة على ثلاثة معاملات كحد أقصى:

```
dest = operand1 op operand2
```

أمثلة:

```
$t0 = 2 3
$t1 = $t0 * 4
x = $t1
```

كل تعبير مركب يُجزأ إلى متغيرات مؤقتة (\$t0, \$t1, ...) وسلسلة من هذه التعليمات البسيطة.

#### أنواع تعليمات TAC المنفّذة

| الرمز  | المعنى             | مثال                |
|--------|--------------------|---------------------|
| ALLOC  | تخصيص متغير/مؤقت   | ALLOC \$t0          |
| ASSIGN | إسناد مباشر        | ASSIGN x, 5         |
| BINARY | عملية ثنائية       | BINARY \$t0, a, b   |
| UNARY  | عملية أحادية       | UNARY \$t1, -, x    |
| PARAM  | تمرير معامل لإجراء | PARAM a             |
| CALL   | استدعاء إجراء      | CALL احسب, 2        |
| LABEL  | علامة مرجعية       | LABEL L0            |
| JUMP   | قفزة غير مشروطة    | JUMP L2             |
| BRANCH | قفزة مشروطة        | BRANCH \$t0, L1, L2 |
| READ   | قراءة من المستخدم  | READ x              |
| PRINT  | طباعة قيمة         | PRINT \$t0          |

مثال شامل: تحويل AST إلى TAC

البرنامج:

```

: صحيح : ج ب, أ, متغير
{
: 2 3 أ :
: 4 * أ = ب :
: ج = أ :
: (ج) اطبع :
}

```

النتيجة TAC تعليمات

```

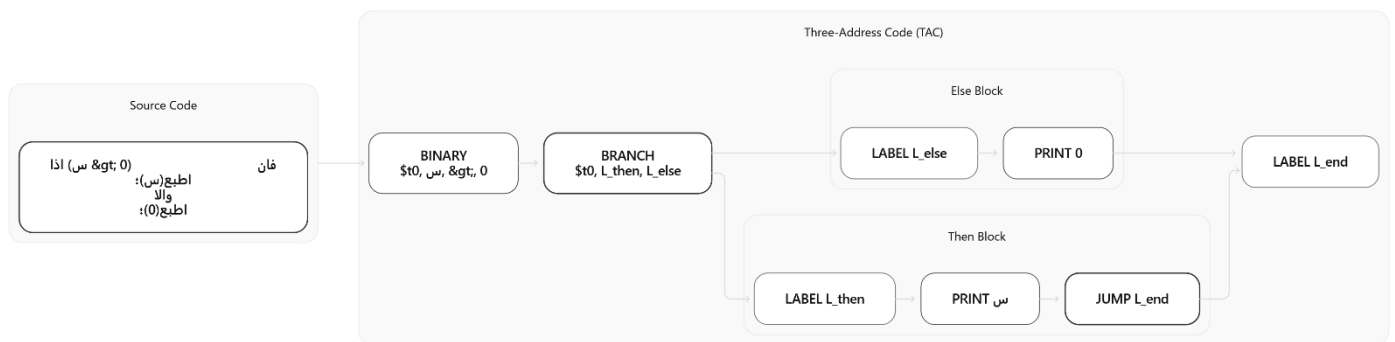
ALLOC أ
ALLOC ب
ALLOC ج
التعبير تجزئة 3, 2, $t0, BINARY
ASSIGN أ, $t0
BINARY $t1, *, 4, أ
ASSIGN ب, $t1
BINARY $t2, , ب, أ
ASSIGN ج, $t2
PRINT ج

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

تمثيل تدفق التحكم في TAC

تُمثّل الفروع والحلقات بعلامات (Labels) وقفزات (Jumps):



شكل توضيحي — reper-intermediate-05

للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

مثال حلقة: كرر (ع = 1 إلى 5) اطبع(ع)؛

ALLOC ع

```

تهيئة 1, ASSIGN
LABEL L_cond
فحص الشرط 5, >=, BINARY $t0
BRANCH $t0, L_body, L_end
LABEL L_body
ع PRINT
الخطوة 1, BINARY $t1
t1$, ASSIGN
عودة للفحص JUMP L_cond
LABEL L_end

```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## الاختبار الذهبي لـ TAC

يحافظ اختبار tac\_golden\_test.c على ناتج محدد لتعبيرات بعينها. مثلاً عند فحص 1 2:

### الناتج المتوقع:

```
BINARY $t0, 1, , 2
```

لا يظهر في المسار الحالي تمرير مستقل لطبي الثوابت (constant folding) — لا يُحسب `t0 = 3` تلقائياً — لذلك لا يُوصف المترجم بوجود optimizer.

## 2.5.2 Typed IR Summary

ما هو في الحقيقة؟

typed\_ir.c لا يُنتج stream تعليمات — بل يُنتج ملخص JSON يُعبئ حقل intermediateRepresentation في استجابة البروتوكول. هدفه: إعطاء الواجهة معلومات منظمة عن البرنامج بعد التحليل.

```

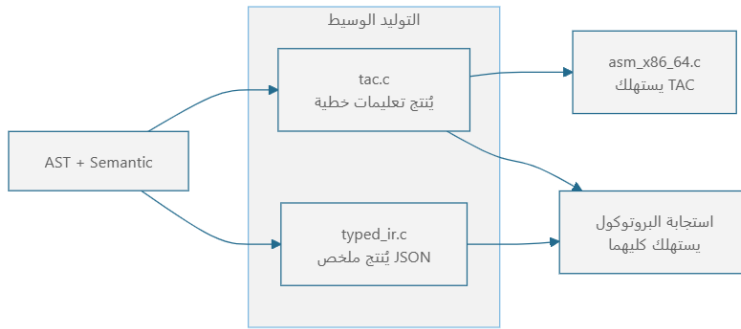
{
  "intermediateRepresentation": {
    "unit": "مثال",
    "name": "مثال",
    "target": "arabicc-1.0",
    "types": ["صحيح", "حقيقي", "منطقي"],
    "symbols": [
      { "name": "أ", "type": "صحيح", "offset": 8 },
      { "name": "ب", "type": "حقيقي", "offset": 16 },
      { "name": "ج", "type": "صحيح", "offset": 24 }
    ],
    "blocks": [
      { "name": "main", "statementCount": 3 }
    ]
  }
}

```

ما يحتوي وما لا يحتوي

| ما لا يحتويه                    | ما يحتويه Typed IR               |
|---------------------------------|----------------------------------|
| تعليمات لكل عملية               | اسم الوحدة والهدف                |
| مخطط كتل التحكم (CFG)           | قائمة أسماء الأنواع المستخدمة    |
| صيغة SSA                        | جدول الرموز مع الأنواع والإزاحات |
| تعليمات LLVM IR كـ intermediate | عدد تعليمات main                 |

المقارنة مع LLVM IR: تمثيل typed قائم على SSA مع تعليمات لكل عملية. Typed IR هنا ملخص بيانات — لا يجوز مساواتهما.



شكل توضيحي — intermediate-repr-05

### 2.5.3 العلاقة بين TAC و Typed IR

#### والمسارات

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره

على التوضيح البصري؛ بل يحدد حدود

كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

النقطة الجوهرية: TAC يتدفق إلى backend، بينما Typed IR يتدفق إلى البروتوكول فقط. هذا فصل مقصود بين ما يحتاجه مولد الشيفرة وما تحتاجه واجهة المستخدم.

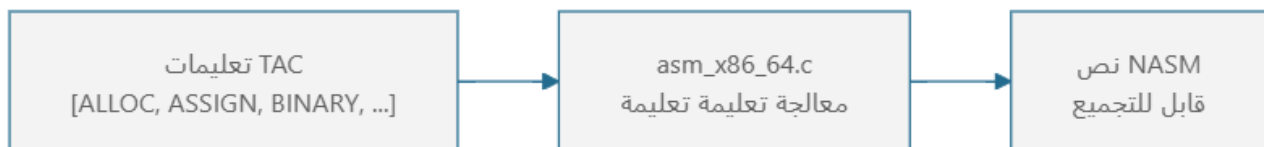
## 2.6 — توليد الشيفرة وبناء الأثر

توليد الشيفرة هو المرحلة الأخيرة في مسار الترجمة الأصلي: تحويل التعليمات الوسيطة (TAC) إلى نص Assembly يفهمه المعالج. ثم — اختياريًا — تجميع ذلك النص وربطه لإنتاج ملف تنفيذي.

### 2.6.1 مولد NASM x86-64

ما يفعله

يستهلك asm\_x86\_64.c تعليمات TAC من tac.c ويُصدر نص NASM لمنصة x86-64:



شكل توضيحي — code-generation-06

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

العمليات المدعومة في NASM backend

| TAC         | نص NASM المُولّد | الملاحظة    |
|-------------|------------------|-------------|
| ASSIGN x, 5 | mov [x], 5       | إسناد مباشر |

|                         |                                              |                            |
|-------------------------|----------------------------------------------|----------------------------|
| حساب صحيح               | mov rax, [a] / add rax, [b] / mov [\$t], rax | BINARY \$t, a, , b         |
| حساب عائِم SSE2         | movsd xmm0, [a] / addsd xmm0, [b]            | BINARY \$t, a, , b (حقيقي) |
| مقارنة                  | cmp rax, rbx / sete al                       | BINARY \$t, a, ==, b       |
| منطقي                   | and rax, rbx                                 | BINARY \$t, a, &&, b       |
| عكس الإشارة             | neg rax                                      | UNARY \$t, -, x            |
| إدخال/إخراج عبر مكتبة C | printf / puts استدعاء                        | PRINT x                    |
| إدخال/إخراج عبر مكتبة C | scanf استدعاء                                | READ x                     |
| علامة في Assembly       | LO:                                          | LABEL LO                   |
| قفزة غير مشروطة         | jmp LO                                       | JUMP LO                    |
| قفزة مشروطة             | cmp rax, 0 / je Lf                           | BRANCH \$t, Lt, Lf         |

مثال: تعليمة حسابية

/\* التعليمة الوسيطة \*/

BINARY \$t0, أ, , ب

/\* NASM المُولِّد (حالة الأعداد الصحيحة) \*/

[أ] mov rax, QWORD

[ب] mov rbx, QWORD

add rax, rbx  
mov QWORD [\$t0], rax

/\* NASM المُولِّد (حالة الأعداد الحقيقية) \*/

[أ] movsd xmm0, QWORD

[ب] addsd xmm0, QWORD

movsd QWORD [\$t0], xmm0

مثال: برنامج كامل

البرنامج:

برنامج مثال:

متغير أ: صحيح؛

{ (أ) اطبع ؛ 3 ؛ 2 = أ }

NASM المُولِّد (مبسَّط):

```
section .data
أ dq 0
$t0 dq 0
```

تنسيق printf للعدد الصحيح; fmt\_d db "%lld", 10, 0

```
section .text
global _start
extern printf
```

```

_start:
3  2 = 1;
    mov rax, 2
    mov rbx, 3
    add rax, rbx
    mov QWORD [$t0], rax
    mov rax, QWORD [$t0]
    mov QWORD [1], rax

```

; اطبع (أ)

```

[1] mov rsi, QWORD
    lea rdi, [rel fmt_d]
    xor eax, eax
    call printf

```

; خروج

```

xor edi, edi
call exit

```

## 2.6.2 حد واضح: تعليمة CALL

NASM backend يرفض تعليمة CALL بصريح العبارة:

"تعليمة استدعاء الإجراء لا تملك ABI في NASM backend"

هذا يعني:

✓ القواعد تقبل تعريف إجراء نحويًا

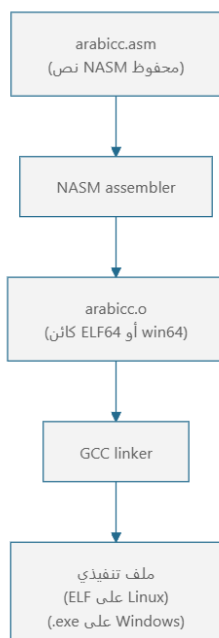
✓ semantic.c يُسجل الإجراء في جدول الرموز

✓ tac.c يُولّد تعليمة CALL

X asm\_x86\_64.c لا يُولّد كود NASM للاستدعاء (غياب اتفاق ABI)

وصف دقيق: "الإجراءات مقبولة لغوياً، وتوليد native code لاستدعائها

غير مكتمل في النسخة الحالية"



## 2.6.3 بناء الأثر (Artifact Builder)

المسار الكامل من Assembly إلى ملف تنفيذي

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

شكل توضيحي — code-generation-06

شروط تفعيل مسار الأثر

1. المستدعي يحدد artifactDirectory صالحاً
2. target = "dart-native"
3. NASM مثبت ومتاح في PATH
4. GCC مثبت ومتاح في PATH
5. مجلد artifacts تحت rootPath (سياسة الأمان)

صيغ المخرج حسب المنصة

| المنصة                | صيغة NASM | المخرج         |
|-----------------------|-----------|----------------|
| Linux / macOS / POSIX | elf64     | ملف ELF تنفيذي |
| Windows               | win64     | ملف (exe) PE   |

يختار artifact\_builder.c الصيغة تلقائياً بناءً على المنصة المكتشفة وقت التشغيل.

2.6.4 سياسة أمان مسار الأثر

```
/* artifact_builder.c يتحقق قبل أي كتابة */
if (!is_path_under_root(artifact_dir, root_path)) {
    return error_diagnostic("backend: مجلد artifacts خارج نطاق المشروع");
}
```

هذا التحقق يمنع الكتابة خارج مساحة المشروع المعتمدة — طبقة مهمة من سياسة المشروع.

2.6.5 ملاحظة: Assembly ليس ملفاً تنفيذياً

ما يظهر في حقل "assembly" بالاستجابة:

نص NASM (سلسلة نصية)

وسيط لا يُشغَّل مباشرة

يصبح ملفاً تنفيذياً فقط بعد:

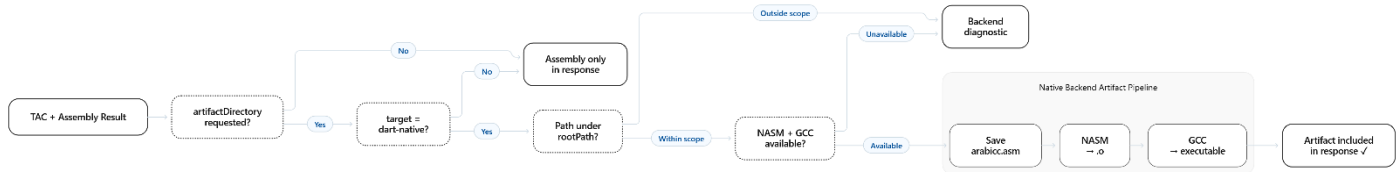
1. حفظه في arabicc.asm

2. تشغيل NASM لإنتاج arabicc.o

3. تشغيل GCC لربط arabicc.o

لذلك: لا تصف نص Assembly بأنه "ملف تنفيذي" أو "artifact".

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكوّن ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.



شكل توضيحي — code-generation-06

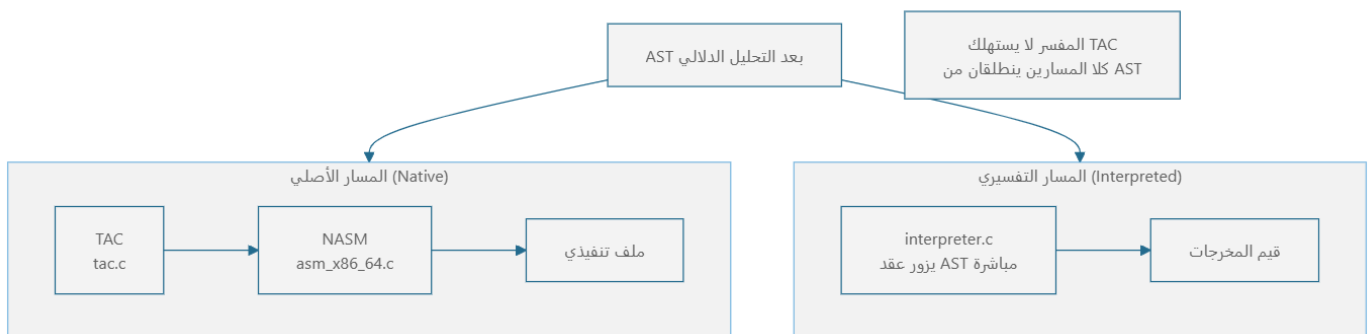
## 2.7 — التنفيذ الداخلي والبروتوكول

### 2.7.1 المفسر (Interpreter)

المفسر يحيل الـ Assembly — لا مكمله

المفسر مسار تنفيذ مستقل تماماً عن مسار NASM. بينما يحوّل مولد NASM الشجرة إلى تعليمات آلة، يُنفذ المفسر الشجرة مباشرة دون توليد شيفرة وسيطة:

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكوّن ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.



شكل توضيحي — runtime-07

التعليمات المدعومة في المفسر

يدعم interpreter.c حالياً هذه الحالات في execute\_statement:

| التعليمة         | الحالة في execute_statement | ملاحظة         |
|------------------|-----------------------------|----------------|
| C_AST_ASSIGNMENT | ✓ مدعومة                    | إسناد قيم      |
| C_AST_PRINT      | ✓ مدعومة                    | طباعة تعبيرات  |
| C_AST_READ       | ✓ مدعومة                    | قراءة من stdin |

|                  |                     |                    |
|------------------|---------------------|--------------------|
| if/else شرط      | ✓ مدعومة            | C_AST_IF           |
| حلقة شرطية       | ✓ مدعومة حارس تكرار | C_AST_WHILE        |
| حلقة عدّية (كرر) | ✓ مدعومة            | C_AST_REPEAT       |
| كتلة تعليمات     | ✓ مدعومة            | C_AST_BLOCK        |
| لا حالة لها      | X غير موجودة        | C_AST_CALL         |
| لا حالة لها      | X غير موجودة        | C_AST_REPEAT_UNTIL |

## حدود ذاكرة المفسر

/\* interpreter.c — جدول القيم \*/

```
#define MAX_VARS 128
```

```
typedef struct {
    char*    name;
    CValue   value;
} VarEntry;
```

```
static VarEntry var_table[MAX_VARS];
```

- الحد الأقصى: 128 متغيراً في وقت التشغيل - حارس while: يوجد حد أقصى للتكرار يمنع حلقات لا نهائية - هذه الحدود معلنة ومقصودة للاستخدام التعليمي

## 2.7.2 البروتوكول: الرسالة بين Flutter و arabicc

### فلسفة البروتوكول

لا يتواصل تطبيق Flutter مع المترجم عبر FFI (استدعاء مكتبة مباشر). بدلاً من ذلك، يُشغّل التطبيق عملية arabicc مستقلة ويتبادل معها رسائل JSON:

```
Flutter App ▶ JSON (stdin) ▶ arabicc ▶ JSON (stdout) ▶ Flutter App
```

هذا القرار يُتيح: - بناء arabicc واختباره بدون Flutter - تشغيل نسخ مختلفة من المترجم دون تعديل التطبيق - عزل أعطال المترجم عن واجهة المستخدم

### بنية طلب البروتوكول (Request)

```
{
  "version": "0.5.0",
  "source": "برنامج مثال\nمتغير أ : صحيح; 5\n{ : اطبع)";
  "mode": "interpret",
  "target": "arabicc-1.0",
  "rootPath": "/home/user/project",
  "artifactDirectory": null,
  "stdin": "",
  "options": {
    "showTokens": true,
    "showAST": true,
    "showTAC": true,
    "showAssembly": true
  }
}
```

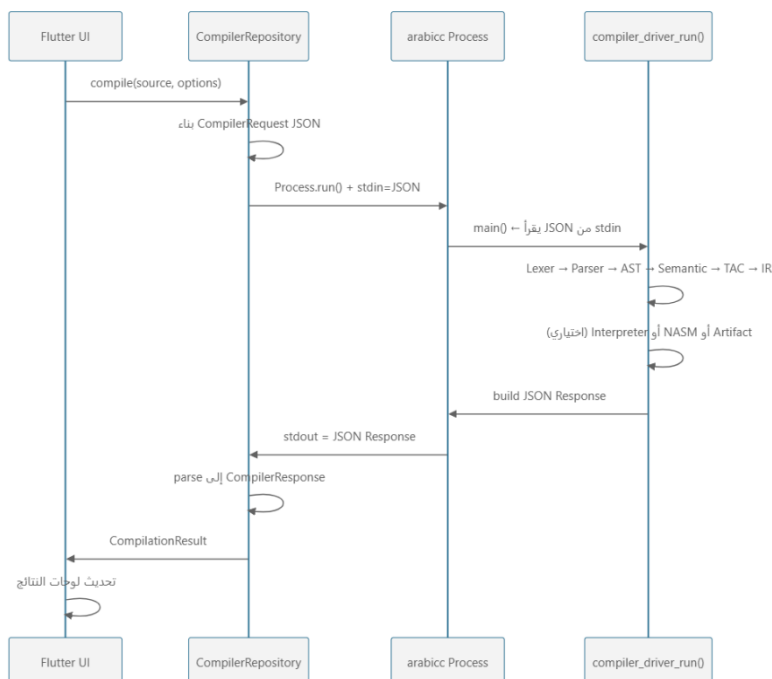
| الحقل   | النوع  | الوظيفة                    |
|---------|--------|----------------------------|
| version | string | إصدار البروتوكول           |
| source  | string | نص البرنامج بالعربية       |
| mode    | enum   | compile أو interpret       |
| target  | string | dart-native أو arabicc-1.0 |

|                              |         |                   |
|------------------------------|---------|-------------------|
| مجلد المشروع الجذري          | string  | rootPath          |
| مسار حفظ الملف التنفيذي      | string? | artifactDirectory |
| مدخلات التنفيذ التفاعلي      | string  | stdin             |
| تفاصيل ما يُدرج في الاستجابة | object  | options           |

### بنية استجابة البروتوكول (Response)

```
{
  "success": true,
  "diagnostics": [],
  "tokens": [...],
  "ast": {...},
  "symbolTable": [...],
  "\n", 5\nPRINT i\nASSIGN i  "threeAddressCode": "ALLOC
  "assembly": "section .data\n...",
  "intermediateRepresentation": {
    "unit": "مثال",
    "symbols": [...],
    "blocks": [{"name": "main", "statementCount": 1}],
    "executionOutput": "5\n",
    "artifacts": null
  }
}
```

| الحقل                      | المصدر             | ملاحظة                         |
|----------------------------|--------------------|--------------------------------|
| success                    | driver             | قد تكون true مع وجود تشخيصات   |
| diagnostics                | جميع المراحل       | قائمة الأخطاء والتحذيرات       |
| tokens                     | lexer.l            | قائمة الرموز مع المواقع        |
| ast                        | parser.y           | شجرة JSON لـ AST               |
| symbolTable                | semantic.c         | جدول الرموز                    |
| threeAddressCode           | tac.c              | نص التعليمات الخطية            |
| assembly                   | asm_x86_64.c       | نص NASM (ليس ملفاً تنفيذياً)   |
| intermediateRepresentation | typed_ir.c         | ملخص JSON                      |
| executionOutput            | interpreter.c      | مخرجات التنفيذ التفسيري        |
| artifacts                  | artifact_builder.c | معلومات الملف التنفيذي إن طُلب |



### دورة الطلب الكاملة

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي

واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.

### شكل توضيحي — runtime-07

#### التشخيصات وتنسيقها

```
{
  "code": "S001",
  "message": "الاسم 'أ' معرّف مسبقاً",
  "severity": "error",
  "location": {
    "line": 3,
    "column": 2,
    "offset": 42
  }
}
```

| نطاق الكود | المرحلة  | أمثلة           |
|------------|----------|-----------------|
| L001       | Lexer    | رمز غير معروف   |
| S001       | Semantic | تعريف مكرر      |
| S002       | Semantic | مرجع غير معرّف  |
| S003       | Semantic | إسناد إلى ثابت  |
| S004       | Semantic | عدم توافق أنواع |
| backend    | code gen | CALL بلا ABI    |

## 2.8 — اختبارات التحقق ومصفوفة القبول وحدود الدليل

التحقق مبدأ جوهري في هذا التوثيق: كل ادعاء تنفيذي يرتبط باختبار أو دليل من المستودع، والفجوات تُعلن صراحةً.

### 2.8.1 الاختبارات الرئيسية في المستودع

| الاختبار               | الملف                                                      | ما يثبته                               |
|------------------------|------------------------------------------------------------|----------------------------------------|
| TAC Golden Test        | packages/compiler_c/tests/tac_golden_test.c                | ناتج TAC لتعبيرات بعينها يطابق المتوقع |
| Assembly Golden Test   | packages/compiler_c/tests/asm_3ac_golden_test.c            | نص NASM المُولّد لحالات TAC محددة      |
| Native Smoke Test      | packages/compiler_c/tests/native_3ac_smoke.sh              | تشغيل المترجم كاملاً وتحقق الخروج      |
| Artifact Security Test | packages/compiler_c/tests/artifact_security_test.c         | التحقق من قيد المسار تحت rootPath      |
| Protocol Smoke         | packages/compiler_c/tests/protocol_smoke.dart              | دورة طلب-استجابة كاملة عبر JSON        |
| Assist Protocol Test   | packages/compiler_contracts/test/assist_protocol_test.dart | نماذج عقود assist                      |
| Process Compiler Test  | test/process_compiler_repository_test.dart                 | تكامّل Flutter مع العملية              |

## 2.8.2 مصفوفة القبول الشاملة

| المجال               | التنفيذ الحالي                    | دليل التحقق                                     | الحدود                                                        |
|----------------------|-----------------------------------|-------------------------------------------------|---------------------------------------------------------------|
| Lexer / Parser / AST | lexer.l parser.y ast.c            | اختبارات البروتوكول والأمثلة                    | تغطية اللغة تتبع grammar المنقّدة — ليست ادعاءً باكتمال اللغة |
| Semantic             | semantic.c                        | اختبارات compiler/protocol                      | قائمة رموز خطية، قدرات محدودة مقارنة بنظام أنواع كامل         |
| TAC                  | tac.c                             | tac_golden_test.c                               | لا تحسين، لا SSA                                              |
| Typed IR             | typed_ir.c                        | نتائج البروتوكول                                | ملخص JSON — ليس stream تعليمات typed                          |
| NASM backend         | asm_x86_64.c                      | asm_3ac_golden_test.c                           | CALL بلا ABI؛ لا دعم كل عمليات اللغة                          |
| Artifact             | artifact_builder.c<br>toolchain.c | native_3ac_smoke.sh<br>artifact_security_test.c | يعتمد على توفر GCC NASM؛ محدود بالمنصة                        |
| Interpreter          | interpreter.c                     | runtime/protocol                                | 128 متغير؛ لا دعم CALL أو REPEAT_UNTIL                        |
| Protocol             | src/protocol/                     | اختبارات العقود                                 | يجب تحديث الجانبين عند تغيير schema                           |
| Flutter Integration  | ProcessCompilerRepository         | اختبارات process repository                     | لا FFI؛ Widgets لا تتعامل مع compiler مباشرة                  |
| CI                   | .github/workflows/ci.yml          | workflow نفسه                                   | راجع jobs: compiler-c, contracts, flutter                     |

## 2.8.3 جدول الادعاءات والأدلة

| الادعاء                         | ما يثبت الدليل                         | ما لا يثبت وحده                             |
|---------------------------------|----------------------------------------|---------------------------------------------|
| Lexer يُعيد رموزاً              | ملفات Flex اختبارات البروتوكول         | اكتمال كل حالات Unicode                     |
| Parser يبني AST                 | ملفات Bison اختبارات parser            | قبول كل صياغة متوقعة بكل حافاتها            |
| Semantic يفحص رموزاً وأنواعاً   | semantic.c اختبارات البروتوكول         | نظام نطاقات كامل أو تحقق من وسائط الإجراءات |
| TAC يُولّد ويُعرض               | tac.c اختبار TAC الذهبي                | وجود تحسين أو بناء SSA                      |
| NASM يُولّد ويُمرّر لـ GCC/NASM | اختبارات smoke Assembly                | دعم كل عمليات اللغة أو كل ABI أو كل منصة    |
| Runtime ينفذ AST                | حالات interpreter.c اختبارات التشغيل   | التطابق الكامل بينه وبين native backend     |
| Typed IR يعرض بيانات typed      | JSON في typed_ir.c                     | تمثيل تعليمات typed كامل أو optimizer       |
| Artifact يُبنى                  | native_3ac_smoke.sh artifact_builder.c | نجاح على كل منصة أو مع كل كود مصدر          |

## 2.8.4 قواعد وصف الدعم

هذه القواعد تمنع التظليل في التوثيق:

1. وجود ملف ≠ اكتمال الميزة وجود `interpreter.c` لا يثبت دعم كل حالات AST
2. وجود حقل في JSON ≠ دعم كامل حقل `assembly` لا يعني أن الكود سيُشغّل بنجاح
3. قبول نحوي ≠ دعم تام قواعد الإجراءات مقبولة نحوياً ودلائياً، لكن NASM يرفضها
4. ربط الادعاء باختبار "الإجراءات مدعومة" اربطها بنتيجة `asm_3ac_golden_test` وليس بوجود `parser.y`
5. الاسم لا يحدد الدور `dart-native` ليس مترجم Dart — هو اسم `target` في `C backend`

## 2.8.5 قيد التحقق في هذه الوثيقة

تحذير منهجي مهم:

لم يُبنَ المترجم ولم تُشغّل اختبارات C في بيئة إعداد هذا التوثيق  
لغياب أدوات CMake / Flex / Bison.

يثبت هذا الفصل:

- ✓ قراءة ملفات التنفيذ (`lexer.l`, `parser.y`, `semantic.c`, ...)
- ✓ قراءة ملفات الاختبارات الموجودة
- ✓ مقارنة المصدر مع الوثائق المجاورة
- X إعادة تشغيل CI أو التحقق من نتائج بناء حديثة

يجب تشغيل هذه الأوامر في بيئة المشروع قبل اعتماد أي تغيير:

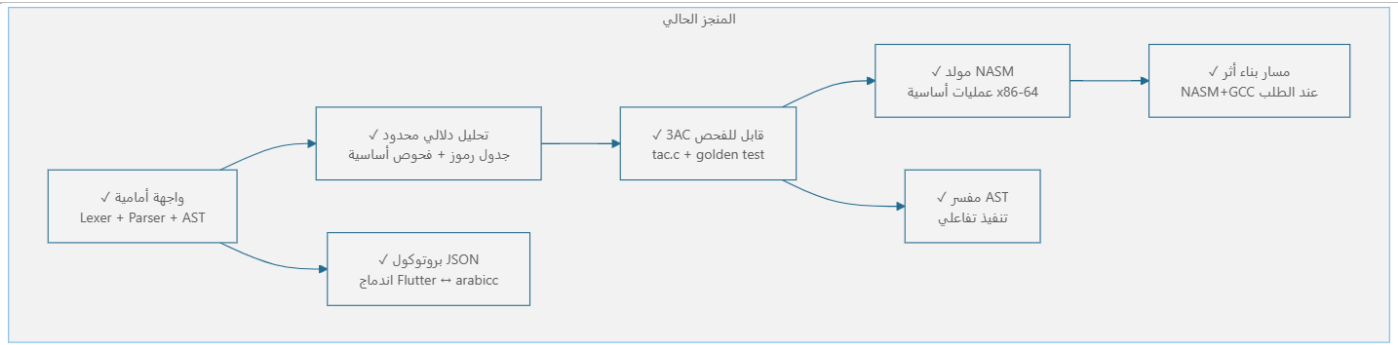
```
cmake --build packages/compiler_c/build/
ctest --test-dir packages/compiler_c/build/
```

## 2.9 — الخلاصة الهندسية والتوصيات

### 2.9.1 ما أنجز فعلاً

البنية الحالية تحتوي العناصر الأساسية لمترجم تعليمي عملي:

يقدم هذا الشكل تمثيلاً بنيوياً أو إجرائياً للعناصر الواردة في القسم، ويُقرأ من خلال اتجاه الأسهم وعلاقات الاعتماد بين العقد. ولا يقتصر دوره على التوضيح البصري؛ بل يحدد حدود كل مكّون ونقطة انتقال البيانات والعقدة التي تتحمل مسؤولية التحقق أو إنتاج المخرج. وبذلك يمكن مطابقة الرسم مع مسار التنفيذ الفعلي واستخدامه كأداة مراجعة للتصميم والاختبارات، مع التمييز بين المسار المنفذ حالياً والمسارات المشروطة بأداة خارجية.



شكل توضيحي — summary-09

## 2.9.2 مقارنة بالمرحلة القياسية

| الملاحظة                  | ما يُعادله في المترجم  | المرحلة القياسية  |
|---------------------------|------------------------|-------------------|
| منجز                      | lexer.l — Flex         | Scanning          |
| منجز                      | parser.y — Bison ast.c | Parsing           |
| منجز جزئياً — بدون نطاقات | semantic.c             | Semantic Analysis |
| منجز — بدون تحسين         | tac.c                  | IR Generation     |
| غير موجودة في المسار      | —                      | Optimization      |
| منجز جزئياً — بدون ABI    | asm_x86_64.c           | Code Generation   |
| منجز — مشروط بالأدوات     | toolchain.c            | Linking           |
| منجز جزئياً — بحدود معلنة | interpreter.c          | Runtime / VM      |

## 2.9.3 التوصيات لرفع نضج المترجم

هذه توصيات تطوير وليست خصائص يقرر هذا الفصل وجودها الآن:

### 1. نطاقات الرموز المتداخلة

الوضع الحالي: قائمة رموز خطية مشتركة

التوصية: مكدس بيئات (scope stack)

لكل إجراء بيئته المستقلة

يدعم: متغيرات محلية، إخفاء الأسماء، التنظيف عند الخروج

### 2. اتفاقية استدعاء الإجراءات (ABI)

الوضع الحالي: CALL تعطي تشخيص "لا ABI"

التوصية: تطبيق System V AMD64 ABI (Linux)

أو Microsoft x64 (Windows)

يشمل: تمرير المعاملات بالسجلات، حفظ السجلات، إدارة الإطار

### 3. توحيد تعريفات التنفيذ

الوضع الحالي: interpreter يدعم حالات مختلفة عن backend

التوصية: قائمة متسقة من التعليمات المدعومة

في كلا المسارين، أو وثيقة تعريف للفجوات

## 4. Typed IR تعليماتي حقيقي

الوضع الحالي: ملخص JSON فقط

التوصية: تعليمات typed لكل عملية

مخطط كتل تحكم (CFG)

ممكناً كمرحلة بين TAC والـ backend

## 5. تمريرات التحسين

التوصية: طي الثوابت (Constant Folding)

إزالة الكود الميت (Dead Code Elimination)

مع اختبارات ذهبية لكل تمرير

## 2.9.4 رسم المسار المستقبلي

المترجم الحالي (تعليمي)

نطاقات متداخلة ABI

Typed IR تعليماتي CFG

تمريرات تحسين SSA

مترجم إنتاجي كامل

## 2.9.5 ملخص الفصل

| الما الذي يثبت          | ما تعلمنا في هذا الفصل                         |
|-------------------------|------------------------------------------------|
| 02-lexical-analysis.md  | كيف يُقطع Lexer النص إلى رموز                  |
| 03-syntax-analysis.md   | كيف يبني Parser شجرة AST                       |
| 04-semantic-analysis.md | كيف يفحص Semantic الرموز والأنواع              |
| 05-intermediate-repr.md | كيف تُولّد تعليمات 3AC وما دور Typed IR        |
| 06-code-generation.md   | كيف يُولّد NASM شيفرة x86-64 ويُنتج ملف تنفيذي |
| 07-runtime.md           | كيف يعمل المفسر والبروتوكول                    |
| 08-verification.md      | ما الذي يثبت الاختبار وما الذي لا يثبت         |

## مرجع اللغة المنفذة في مترجم C

هذا مرجع تنفيذي لقواعد lexer.y و parser.y الحالية، وليس مواصفة مستقلة عن الشيفرة. < يصف المسار الفعّال من نقطة البداية program إلى full\_program.

## 1. الرموز المحبوزة الكاملة

| المعنى       | الألفاظ المقبولة حرفياً في lexer | رمز Bison   |
|--------------|----------------------------------|-------------|
| بدء البرنامج | برنامج                           | TOK_PROGRAM |
| ثابت         | ثابت                             | TOK_CONST   |
| تعريف نوع    | نوع                              | TOK_TYPE    |
| تعريف متغير  | متغير                            | TOK_VAR     |

|                 |              |               |
|-----------------|--------------|---------------|
| TOK_PROCEDURE   | إجراء, إجراء | إجراء         |
| TOK_BY_VALUE    | بالقيمة      | تمرير بالقيمة |
| TOK_BY_REF      | بالمرجع      | تمرير بالمرجع |
| TOK_PRINT       | اطبع         | طباعة         |
| TOK_READ        | اقرأ, اقرأ   | قراءة         |
| TOK_IF          | إذا, إذا     | شرط — رأس     |
| TOK_THEN        | فإن          | شرط — ثم      |
| TOK_ELSE        | وإلا, وإلا   | شرط — وإلا    |
| TOK_WHILE       | طالما        | حلقة شرطية    |
| TOK_DO          | استمر        | حلقة — استمر  |
| TOK_REPEAT      | كرر          | حلقة عدّية    |
| TOK_FROM        | من           | مدى — من      |
| TOK_TO          | إلى          | مدى — إلى     |
| TOK_STEP        | أضف, أضف     | مدى — خطوة    |
| TOK_UNTIL       | حتى          | تكرار — حتى   |
| TOK_AGAIN       | أعد, أعد     | تكرار — أعد   |
| TOK_TYPE_INT    | صحيح         | نوع صحيح      |
| TOK_TYPE_REAL   | حقيقي        | نوع حقيقي     |
| TOK_TYPE_BOOL   | منطقي        | نوع منطقي     |
| TOK_TYPE_CHAR   | حرفي         | نوع حرفي      |
| TOK_TYPE_STRING | خيط رمزي     | نوع سلسلة     |
| TOK_ARRAY       | قائمة        | مصفوفة        |
| TOK_RECORD      | سجل          | سجل           |
| TOK_TRUE        | صح           | صحيح منطقي    |
| TOK_FALSE       | خطأ          | خاطئ منطقي    |

ملاحظة الإملاء: ليست كل بدائل الهمزة موجودة. إلى تُكتب بلا همزة. اطبع بلا بديل.

## 2. الفواصل والعوامل

| العامل/الرمز | النوع         | الملاحظة      |
|--------------|---------------|---------------|
| ;/           | فاصلة منقوطة  | كلاهما يعيد ; |
| ,/           | فاصلة         | كلاهما يعيد , |
| ==           | مساواة        |               |
| != أو !=     | لا مساواة     | كلاهما TOK_NE |
| <, >         | مقارنة        |               |
| >= أو >=     | أصغر أو يساوي |               |
| <= أو <=     | أكبر أو يساوي |               |
| &&           | و منطقي       |               |
| \            | \             | أو منطقي      |
| !            | نفي منطقي     |               |
| , -, *, /, % | حساب          |               |

|              |            |          |
|--------------|------------|----------|
| ارتباط يميني | أس         | ^        |
| TOK_INT_DIV  | قسمة صحيحة | \\ أو \\ |

### 3. القواعد النحوية الكاملة (EBNF)

(\* ===== نقطة الدخول ===== \*)

```
" declarations block "." ;:" IDENT "برنامج" program ::= "
```

(\* ===== التصريحات ===== \*)

```
" } ;:declarations ::= { declaration "
declaration ::= constant_decl
                | type_decl
                | variable_decl
                | procedure_decl ;
```

```
" IDENT "=" expression ;:constant_decl ::= "
" IDENT "=" type ;:type_decl ::= "
" name_list ":" type ;:variable_decl ::= "
name_list ::= IDENT { "," IDENT } ;
```

(\* ===== الإجراءات ===== \*)

```
" block ;:" IDENT "(" [ parameters ] ")" "اجراء" procedure_decl ::= "
" parameter_group } ;:parameters ::= parameter_group { "
" ) name_list ":" type ;:parameter_group ::= ( "
```

(\* ===== الأنواع ===== \*)

```
"type صحيح" ::= "
```

"| حقيقي"

"| منطقي"

"| حرفي"

"| خيط رمزي"

```
| IDENT
```

"| قائمة" (INT | IDENT) "[" "من" type

```
" ] " ;:" field } [ ":" "{" field { "سجل"
field ::= IDENT ":" type ;
```

(\* ===== الكتلة والتعليمات ===== \*)

```
block ::= "{" statements "}" ;
" ] } ;:" | if_statement [ ":"statements ::= { statement "
statement ::= assignment
                | read
                | print
                | call
                | if_statement
                | while_statement
                | counted_repeat
                | repeat_until
                | block ;
```

(\* ===== التعليمات المفردة ===== \*)

```
assignment ::= access "=" expression ;
" "(" access ")" "اقرأ" read ::= "
" "(" [ arguments ] ")" "اطبع" print ::= "
call ::= IDENT "(" [ arguments ] ")" ;
arguments ::= expression { "," expression } ;
```

```

(*) ===== تعليمات التحكم ===== *)
" statement ; استمر ; " (" expression ") " طالما while_statement ::= "

" expression الى ; " (" IDENT "=" expression "كرر counted repeat ::= "
; statement (" [ expression ( "أضف | "أضف " ) ]

" (" expression ") " حتى ; statement " أعد | " أعد repeat_until ::= ( "

"; statement " فان " (" expression ") " اذا if_statement ::= "
"; statement " فان " (" expression ") " اذا |
"; statement " والا "
"; statement " فان " (" expression ") " اذا |
; if_statement " والا "

(*) ===== الوصول والتعبيرات ===== *)
access ::= IDENT { "[" expression "]" | "." IDENT } ;

expression ::= expression "|" expression
              | expression "&&" expression
              | expression compare_op expression
              | expression " " expression
              | expression "-" expression
              | term ;

term ::= term "*" factor
       | term "/" factor
       | term "%" factor
       | term INT_DIV factor
       | term "^" factor
       | factor ;

factor ::= INT | REAL | STRING | CHAR
        | "خطأ"
        | access
        | "!" factor
        | " " factor
        | "-" factor
        | "(" expression ")" ;

compare_op ::= "==" | "!=" | "<" | ">" | "<=" | ">=" ;

```

#### 4. الأسبقية والارتباط

| الترتيب (أدنى أعلى) | العوامل              | الارتباط     |
|---------------------|----------------------|--------------|
| 1                   | \                    | يساري        |
| 2                   | &&                   | يساري        |
| 3                   | ==, !=, <, >, <=, >= | يساري        |
| 4                   | , -                  | يساري        |
| 5                   | *, /, %, INT_DIV     | يساري        |
| 6                   | ^                    | يميني        |
| خاص                 | UPLUS, UMINUS, !     | يميني (معلن) |

## 5. القواعد المعجمية

```
lexer.
ARABIC_CHAR نطاقات UTF-8 المحددة لأحرف العربية في.
IDENT (ARABIC_CHAR | [a-zA-Z_]) (ARABIC_CHAR | [a-zA-Z0-9_])*
INT [0-9]
REAL [0-9] '.' [0-9]
STRING '"' ([^"\\] | '\\.)* '"'
CHAR '"' . '"' | "'" . "'"
COMMENT ' ' [^\n]* (
Bison) لا يُعاد ل.
UNKNOWN أي رمز غير مطابق تشخيص L001
```

مخطط بنية نصي — يوضح التسلسل والعلاقات بين العناصر

## 6. أمثلة توافقية

(\* برنامج بسيط \*)

برنامج مثال؛

متغير س : صحيح؛

{

س = 2 \* 3 \* 4؛

إذا (س <= 10) فإن اطبع(س)؛ وإلا اطبع(0)؛

}

.

(\* حلقة عدّية \*)

برنامج عدّ؛

متغير ع : صحيح؛

{

كرر (ع = 1 إلى 5 أضع 1) اطبع(ع)؛

}

.

(\* سجل \*)

برنامج نقطة؛

نوع نقطة\_ثنائية = سجل { س : حقيقي ؛ ص : حقيقي } ؛

متغير ن : نقطة\_ثنائية؛

{

ن.س = 14.3؛

ن.ص = 1.27؛

اطبع(ن.س)؛

}

.

## 7. حدود مثبتة من المصدر

1. القواعد القديمة (statement, expr) موجودة في parser.y لكن نقطة البداية تقود إلى full\_program — لا تخط بينهما 2. طول المصفوفة بمعرّف مقبول نحوياً، لكن CTypeSpec.length = 0 في ذلك الفرع 3. وجود قواعد للإجراءات لا يثبت دعم ABI في CALL — backend يعطي تشخيصاً 4. القواعد المعجمية والنحوية وحدها لا تثبت اكتمال التحليل الدلالي أو التنفيذ

## مصادر التحقق

| المصدر                             | الوصف                    |
|------------------------------------|--------------------------|
| packages/compiler_c/src/lexer.l    | القواعد المعجمية الفعلية |
| packages/compiler_c/src/parser.y   | القواعد النحوية الفعلية  |
| packages/compiler_c/include/ast.h  | أنواع عقد AST            |
| packages/compiler_c/src/semantic.c | التحليل الدلالي          |

## المراجع — الفصل الثاني

### المراجع الأكاديمية

Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, \*Compilers: Principles, Techniques, and Tools\* (2nd [1] ed.), Addison-Wesley, 2006. — المرجع الأساسي لمراحل المترجم: Lexer, Parser, Semantic, IR, Code Generation. — Des Watson, \*A Practical Approach to Compiler Construction\*, Springer, 2017. <<https://doi.org/10.1007/978-3-319-52789-5>> [2] يصف التقسيم التعليمي لمراحل التحليل المعجمي والنحوي والدلالي والتمثيلات الوسيطة وتوليد الشيفرة.

### توثيق الأدوات المستخدمة

[3] <Flex Manual — Matching, <https://westes.github.io/flex/manual/Matching.html>> — مبدأ أطول تطابق وأسبقية القاعدة في Flex.

[4] <Flex Manual — Patterns, <https://westes.github.io/flex/manual/Patterns.html>> — تعريف أنماط الرموز والتحرير.

[5] <GNU Bison Manual — Specifying Operator Precedence, [https://www.gnu.org/software/bison/manual/html\\_node/Precedence.html](https://www.gnu.org/software/bison/manual/html_node/Precedence.html)> — توجيهات الأسبقية والارتباط لحسم تعارضات shift/reduce.

[6] LLVM IR — LLVM Language Reference Manual, <<https://llvm.org/docs/LangRef.html>> — LLVM Project تمثيل typed قائم على SSA؛ للمقارنة مع Typed IR المحلي.

[7] <LLVM Project — Writing an LLVM Backend, <https://llvm.org/docs/WritingAnLLVMBackend.html>> — فصل مسؤوليات الخلفية المعتمدة على الهدف عن تحليل اللغة.

[8] <NASM Manual — Output Formats, <https://www.nasm.us/doc/nasm09.html>> — صيغ elf64 لـ NASM، و POSIX، و win64 لـ Windows.

— <GNU Compiler Collection — Options for Linking, <<https://gcc.gnu.org/onlinedocs/gcc/Link-Options.html> [9]  
 خيارات الربط المستخدمة مع الملف الهدف في toolchain.c.

## مصادر التحقق من المستودع

| الوصف                       | الملف                                               |
|-----------------------------|-----------------------------------------------------|
| Flex — القواعد المعجمية     | packages/compiler_c/src/lexer.l                     |
| Bison — القواعد النحوية     | packages/compiler_c/src/parser.y                    |
| أنواع عقد AST               | packages/compiler_c/include/ast.h                   |
| التحليل الدلالي             | packages/compiler_c/src/semantic.c                  |
| مولد 3AC                    | packages/compiler_c/src/ir/tac.c                    |
| ملخص Typed IR               | packages/compiler_c/src/ir/typed_ir.c               |
| مولد NASM                   | packages/compiler_c/src/backend/x86_64/asm_x86_64.c |
| بناء الأثر                  | packages/compiler_c/src/backend/artifact_builder.c  |
| إدارة أدوات البناء          | packages/compiler_c/src/backend/toolchain.c         |
| المفسر                      | packages/compiler_c/src/runtime/interpreter.c       |
| driver الرئيسي              | packages/compiler_c/src/driver/compiler_driver.c    |
| اختبار TAC الذهبي           | packages/compiler_c/tests/tac_golden_test.c         |
| اختبار Assembly الذهبي      | packages/compiler_c/tests/asm_3ac_golden_test.c     |
| اختبار دخاني للتنفيذ الأصلي | packages/compiler_c/tests/native_3ac_smoke.sh       |
| اختبار أمان مسار الأثر      | packages/compiler_c/tests/artifact_security_test.c  |
| عقود البروتوكول             | packages/compiler_contracts/                        |
| بوابات CI                   | .github/workflows/ci.yml                            |

# التنفيذ والصور

## واجهه المحرر

The screenshot shows the Arabic360 IDE interface. The main editor displays a list of rules in Arabic, including:

- برنامج شامل:
- ثابت حد = 4:
- ثابت تحية = "أبداً":
- نوع اعداد = قائمة(جد) من صحيح:
- متغير قائمة\_الاعداد: اعداد:
- متغير طالب: سجل\_طالب:
- متغير ا, مجموع: قيمة: صحيح:
- متغير متوسط: حقيقي:
- متغير جاهز: منطقي:
- إجراء احسب(القيمة رقم: صحيح: بالمرجع ناتج: صحيح):
- ناتج = رقم \* 2:
- قائمة\_الاعداد[0] = 2:
- قائمة\_الاعداد[1] = 4:
- قائمة\_الاعداد[2] = 6:
- قائمة\_الاعداد[3] = 8:
- طالب اسم = "سارة":
- طالب درجة = 91.5:

The sidebar on the right shows the project structure for 'arabic360', including files like '01\_basics.arb', '02\_composite\_types.arb', '03\_arithmetic.arb', '04\_io.arb', '05\_conditions.arb', '06\_loops.arb', '07\_procedures.arb', '08\_syntax\_error.arb', '09\_semantic\_error.arb', '10\_all\_rules.arb', 'main.arb', 'اختيارات.md', and 'اليمين.arb'.

At the bottom, there is a status bar showing 'تعديلات غير محفوظة: 0' and the file path 'C:\Users\Thinkpad\Downloads\main'.

## فتح ملف

The screenshot shows the 'Welcome' screen of the Arabic360 IDE. The main area displays the text 'Arabic360' and 'لم يتم فتح مجلد بعد' (No folder opened yet). Below this, there are three buttons: 'فتح مجلد' (Open Folder), 'فتح ملف' (Open File), and 'ملف جديد' (New File).

The sidebar on the right shows the 'Explorer' panel with the text 'No Folder Opened' and a message: 'لم يتم فتح مجلد بعد.' (No folder opened yet). Below this, there is a button 'فتح مجلد' (Open Folder) and a message: 'فتح مجلد سيعرض شجرة المشروع هنا. بينما تبقى الملفات المفتوحة مستقلة عن Workspace.' (Opening a folder will display the project tree here. While the files remain open, they are independent of the Workspace).

At the bottom, there is a status bar showing 'تعديلات غير محفوظة: 0' and the file path 'C:\Users\Thinkpad\Downloads\main'.

نتائج ال token

acsys360

ain.arb X basics.arb\_01 X arithmetic.arb\_03 X io.arb\_04 X syntax\_error.arb\_08 X all\_rules.arb\_10 X

main main < all\_rules.arb\_10 >

برنامج شامل؛ 1

نتائج الترجمة

Artifact Assembly Typed IR 3AC Semantic Symbol Table Syntax Tree Tokens التنفيذ الأخطاء

جدول Tokens

| # | النوع       | النص   | السطر | العمود | الطول | الملف                                  |
|---|-------------|--------|-------|--------|-------|----------------------------------------|
| 1 | keyword     | برنامج | 1     | 1      | 6     | ...C:\Users\Thinkpad\Downloads\main\10 |
| 2 | identifier  | شامل   | 1     | 8      | 4     | ...C:\Users\Thinkpad\Downloads\main\10 |
| 3 | punctuation | ;      | 1     | 12     | 1     | ...C:\Users\Thinkpad\Downloads\main\10 |
| 4 | keyword     | ثابت   | 2     | 1      | 4     | ...C:\Users\Thinkpad\Downloads\main\10 |
| 5 | identifier  | حد     | 2     | 6      | 2     | ...C:\Users\Thinkpad\Downloads\main\10 |
| 6 | operator    | =      | 2     | 9      | 1     | ...C:\Users\Thinkpad\Downloads\main\10 |
| 7 | integer     | 4      | 2     | 11     | 1     | ...C:\Users\Thinkpad\Downloads\main\10 |
| 8 | punctuation | ;      | 2     | 12     | 1     | ...C:\Users\Thinkpad\Downloads\main\10 |

تعديلات غير محفوظة: 0

C:\Users\Thinkpad\Downloads\main

محرر العربية

مستكشف المشروع

main

arabic360

01\_basics.arb

02\_composite\_types.arb

03\_arithmetic.arb

04\_io.arb

05\_conditions.arb

06\_loops.arb

07\_procedures.arb

08\_syntax\_error.arb

09\_semantic\_error.arb

10\_all\_rules.arb

main.arb

README\_اختيارات.md

اليمين.arb

شجرة الاعراب

acsys360

ain.arb X basics.arb\_01 X arithmetic.arb\_03 X io.arb\_04 X syntax\_error.arb\_08 X all\_rules.arb\_10 X

main main < all\_rules.arb\_10 >

برنامج شامل؛ 1

نتائج الترجمة

Artifact Assembly Typed IR 3AC Semantic Symbol Table Syntax Tree Tokens التنفيذ الأخطاء

عرض الشجرة

| العقدة                                 | المستوى | البيانات                            |
|----------------------------------------|---------|-------------------------------------|
| program                                | 0       | name: شامل                          |
| declarations[0] · constant_declaration | 1       | name: حد                            |
| value · literal                        | 2       | literalKind: integer, value: 4      |
| declarations[1] · constant_declaration | 1       | name: تجبة                          |
| value · literal                        | 2       | literalKind: string, value: "ابتدا" |
| declarations[2] · type_declaration     | 1       | name: اعداد                         |
| type · array                           | 2       | length: 0                           |
| elementType · named                    | 3       | name: صحيح                          |

محرر العربية

مستكشف المشروع

main

arabic360

01\_basics.arb

02\_composite\_types.arb

03\_arithmetic.arb

04\_io.arb

05\_conditions.arb

06\_loops.arb

07\_procedures.arb

08\_syntax\_error.arb

09\_semantic\_error.arb

10\_all\_rules.arb

main.arb

README\_اختيارات.md

اليمين.arb

جدول الرموز

acsys360

ain.arb X basics.arb\_01 X arithmetic.arb\_03 X io.arb\_04 X syntax\_error.arb\_08 X all\_rules.arb\_10 X

main < all\_rules.arb\_10 <>

برناميج شامل؛ 1

نتائج الترجمة

Artifact Assembly Typed IR 3AC Semantic Symbol Table Syntax Tree Tokens التنفيذ الأخطاء

جدول الرموز

| الاسم         | الصف     | النوع   | الملف            | السطر | العمود |
|---------------|----------|---------|------------------|-------|--------|
| حد            | variable | متغير   | all_rules.arb_10 | 2     | 6      |
| تحية          | variable | خط رمزي | all_rules.arb_10 | 3     | 6      |
| اعداد         | variable | نوع     | all_rules.arb_10 | 4     | 5      |
| سجل طلب       | variable | نوع     | all_rules.arb_10 | 5     | 5      |
| قائمة الاعداد | variable | اعداد   | all_rules.arb_10 | 6     | 7      |
| طالب          | variable | سجل طلب | all_rules.arb_10 | 7     | 7      |
| i             | variable | متغير   | all_rules.arb_10 | 8     | 7      |
| مجموع         | variable | متغير   | all_rules.arb_10 | 8     | 7      |

تعديلات غير محفوظة: 0

C:\Users\Thinkpad\Downloads\main

محرر العربية

مستكشف المشروع

main

arabic360

01\_basics.arb

02\_composite\_types.arb

03\_arithmetic.arb

04\_io.arb

05\_conditions.arb

06\_loops.arb

07\_procedures.arb

08\_syntax\_error.arb

09\_semantic\_error.arb

10\_all\_rules.arb

main.arb

README\_اختيارات.md

اليمن.arb

نتائج التحليل الدلالي لخطأ منطقي

acsys360

ain.arb X basics.arb\_01 X arithmetic.arb\_03 X io.arb\_04 X syntax\_error.arb\_08 X all\_rules.arb\_10 X

main < loops.arb\_06 <>

متغير مجموع، i: صحيح؛  
}  
مجموع = "احمد";  
كرر (i = 1 إلى 5 أضع 1) مجموع = مجموع + i;

نتائج الترجمة

Artifact Assembly Typed IR 3AC Semantic Symbol Table Syntax Tree Tokens التنفيذ الأخطاء

JSON

Semantic

[  
{  
"severity": "error",  
"phase": "semantic",  
"code": "SEM001",  
"message": "عدم توافق نوع الإسناد للرمز «مجموع»: المتوقع «صحيح» والمستلم «خط رمزي»",  
"span": {  
"sourcePath": "C:\\Users\\Thinkpad\\Downloads\\main\\06\_loops.arb",  
"offset": 43,  
"line": 4,  
"column": 3,  
"length": 5  
}  
}  
]

تعديلات غير محفوظة: 1

C:\Users\Thinkpad\Downloads\main

محرر العربية

مستكشف المشروع

main

arabic360

01\_basics.arb

02\_composite\_types.arb

03\_arithmetic.arb

04\_io.arb

05\_conditions.arb

06\_loops.arb

07\_procedures.arb

08\_syntax\_error.arb

09\_semantic\_error.arb

10\_all\_rules.arb

main.arb

README\_اختيارات.md

اليمن.arb



## الجدول الرباعي

acsys360

محرر العربية

مستكشف المشروع

main

main

arabic360

01\_basics.arb

02\_composite\_types.arb

03\_arithmetic.arb

04\_io.arb

05\_conditions.arb

06\_loops.arb

07\_procedures.arb

08\_syntax\_error.arb

09\_semantic\_error.arb

10\_all\_rules.arb

main.arb

README\_اختيارات.md

اليمن.arb

نتائج الترجمة

Artifact Assembly Typed IR 3AC Semantic Symbol Table Syntax Tree Tokens التنفيذ الأخطاء

3AC جدول

| result | arg2 | arg1 | op    | # |
|--------|------|------|-------|---|
| مجموع  |      | صحيح | ALLOC | 1 |
|        | i    | صحيح | ALLOC | 2 |
|        | z    | صحيح | ALLOC | 3 |
| مجموع  |      | 0    | =     | 4 |
|        | i    | 1    | =     | 5 |
|        | L0   |      | label | 6 |

## الشفرة التجميعية

acsys360

محرر العربية

مستكشف المشروع

main

main

arabic360

01\_basics.arb

02\_composite\_types.arb

03\_arithmetic.arb

04\_io.arb

05\_conditions.arb

06\_loops.arb

07\_procedures.arb

08\_syntax\_error.arb

09\_semantic\_error.arb

10\_all\_rules.arb

main.arb

README\_اختيارات.md

اليمن.arb

نتائج الترجمة

Artifact Assembly Typed IR 3AC Semantic Symbol Table Syntax Tree Tokens التنفيذ الأخطاء

Assembly

تعليمات التجميع

```

push rax
mov rax, 1
mov rcx, rax
pop rax
add rax, rcx
mov [rbp-64], rax
mov rax, [rbp-64]
mov [rel global_2], rax
mov rax, [rel global_2]
mov rdx, rax
lea rcx, [rel fmt_int]
xor eax, eax
call printf
jmp L2
L3:
L4:
mov rax, [rel global_0]
push rax
mov rax, 1
mov rcx, rax
pop rax
sub rax, rcx
mov [rbp-72], rax

```